

GwenLaya v4: a pre-registered study of answering only when a check can back the answer

Interim report: registered design, deviations, a claim audit, and exploratory full-E baselines on TPU, a math gate, calibrated abstention and a 9B tier;
no completed pre-registered test

Xavier Callens
AutoevolveAI / ANSE open-source project

9 October 2026 (revision 3.5.0, branch `feat/math-gate`; supersedes 3.4.0)

Abstract

GwenLaya is a system design in which a small open model (Qwen3.5 family) answers a programming or mathematics question only when an executed check or a calibrated confidence score supports the answer, and otherwise abstains or escalates to a larger tier. We pre-registered fifteen hypotheses (two primary: fewer confident-wrong answers than log-probability selective prediction at matched coverage, H1; lower cost per correct answered item than the largest tier with the gate, H3) before any GwenLaya evaluation data were generated. **No pre-registered hypothesis test has been completed.** Both primaries need the Laya calibrator and a separate calibration split C, and neither exists yet.

Revision 3.4.0 added *exploratory, unregistered* baselines on the full E set (1,536 tasks: Python 529, Rust 507, math 500). Revision 3.5.0 adds three further exploratory analyses on the same E set, all post hoc: (A10) the registered program-of-thought math gate, which 3.4.0 reported as not implemented; (A11) cross-fitted calibrated abstention; (A12) a Qwen3.5-9B tier on a four-chip TPU slice with gate cascades. Models: Qwen2.5-Coder-1.5B-Instruct, Qwen3.5-2B, 4B and 9B. All used greedy decoding with one sample, in bf16 under vLLM on TPU v5e, and all outputs were scored locally in a sandbox. E is burned for the registered H1/H3 (Section 7). The 3.4.0 analysis plan was committed after generation and as the 2B/4B scoring started, before any scored outcome was inspected but after smaller descriptive runs on 80 tasks had shown approximate pass rates. The A10, A11 and A12 addenda were each written before their results were computed, but after the E baselines had been seen. **This revision supersedes the 3.4.0 statement that the math gate has coverage 0:** that was an implementation gap in our checker, now closed (Section 6.6).

Pooled pass rates on the hidden checks were 0.423 (2B), 0.606 (4B), 0.653 (9B) and 0.482 (Coder). Most of the findings are negative or null: (i) Raw $\exp(\text{mean log-prob})$ confidence is badly calibrated. On the 4B its mean is 0.927–0.938 per domain, while only 0.529–0.675 of answers pass. (ii) A gate cascade (2B, then an executed gate, then the 4B if the gate does not verify) is “GwenLaya without Laya” and is not a registered arm. With the program-of-thought math gate and its generation cost charged, it gains no accuracy over always using the 4B: the pooled difference is -0.001 (CI -0.013 to 0.012). Pooled, it costs about twice the accelerator time (cost ratio 2.021, CI 1.933–2.111, with the first compilation chunk; 2.023, CI 1.912–2.134, without it), because on math it pays for programs and usually for both tiers. On Python alone it breaks even with warm-up (0.997) and is about 10% cheaper without it (0.900), at a 2.6-point accuracy cost. Cost is configuration dependent and these ratios are not robust. (iii) The program-of-thought math gate (A10) verifies 0.454 of the 4B’s math answers (CI 0.410–0.498) with precision 0.943 (CI 0.911–0.971), just below the 0.95 flag point of the registration, and the CI includes it. Its confident-wrong rate over all math tasks is 0.026, against 0.388 when everything is answered. (iv) A tier gap does change the cascade result (A12). Using a fixed reading rule written before the 9B results (a cascade pays off only if it is cheaper than always-9B in both the with- and without-warm-up readings and its accuracy CI does not exclude zero on the harmful side), 4B→9B pays off for Python and Rust but not

for math (programs are charged for each tier consulted) and not pooled (cost ratio 1.381 with warm-up and 1.724 without, although pooled accuracy is higher by 0.037). 2B→9B on Python fails on accuracy (-0.042 , CI -0.068 to -0.015). The 9B’s effective engine concurrency was not measured, so its cost is less certain. (v) A cross-fitted logistic score (A11) that includes the gate verdict ranks and calibrates better than log-prob alone (4B pooled AUROC 0.931 against 0.708). Because it uses the gate as a feature, that is a calibration gain, not independent evidence. Its ECE bootstrap intervals sometimes exclude their own point estimates, a known bias of percentile bootstrap for ECE.

The most favourable observation remains the 4B gate. Answering only when the executed gate verifies gave fewer confident-wrong answers than a log-prob threshold at the same coverage: -0.076 for Python, -0.112 for Rust and, with the new math gate, -0.084 for math. The baseline answers the k most confident tasks, with k equal to the gate’s answered count in the same domain. Several caveats apply. The gate runs tests or programs that the log-prob does not see. k is matched on E, not taken from a threshold frozen on a separate split. The benchmarks are public and were probably seen in pretraining. There is a single greedy sample. This comparison is not H1, and nothing here supports H1 or H3 or says anything about Laya (no Laya head was trained).

A serial re-check of all 1,536 non-passing code rows gave 3 flips. Two of them (one task, for the 2B and the 4B) changed the headline. The third is a nondeterministic candidate and stays FAILED. A re-run of all 1,572 passing rows changed none.

This revision also keeps the corrections of 3.4.0 (Section 2). The recorded GCP spend is \$4.91, a list-price estimate from 11 ledger lines (\$2.05 of it for the 9B smoke test and full run). It says nothing about the system.

1 Retraction and the v3 gate failure, stated first

Retraction. Revision 3.1.0 of the predecessor report [2] claimed that the low-tier optimizer improved pass rates by at least $+8.0$ percentage points. That claim rested on a hand-written results file with invented numbers (A0 33%, A3 100%). The file was fabricated, it has been deleted, and it is not evidence (`results/PREREGISTRATION.md`). The claim is retracted. Nothing in this paper depends on it.

The measured v3 gate failed. The pre-registered v3 gate was $A3 - A0 \geq +8.0$ points pass@1 on MBPP-sanitized with `qwen2.5-coder:1.5b`. Both measured runs **FAIL** that gate:

- $n = 60$, CPU according to the file name and README (the file itself records hardware “unspecified”): $A3 - A0 = +6.67$ points, 95% CI $[1.67, 13.33]$, 4 gained / 0 lost, exact McNemar $p = 0.125$; verdict FAIL (`results/results_n60_cpu.json`).
- $n = 257$, L4 GPU: A0 64.2%, A3 71.98%, $A3 - A0 = +7.78$ points, 95% CI $[4.67, 11.28]$, 20 gained / 0 lost; verdict FAIL against $+8.0$. This file’s `git_commit` field is empty (`results/results_n257_l4_gpu.json`).

At $n = 60$ the difference is not significant by the exact McNemar test ($p = 0.125$, 4/0 discordant pairs). The percentile-bootstrap interval there is not informative. When every discordant pair favours A3, a resample can reach $\Delta \leq 0$ only by drawing no discordant item at all, so the lower bound is positive almost by construction. Only the $n = 257$ run supports a nonzero effect (20/0 discordant; the file rounds the exact McNemar p to 0.0, i.e. $p < 0.001$). Both runs missed the registered $+8.0$ threshold, and we report both as FAIL.

The same file shows the v3 gate weakness that motivates v4. Of the answers the gate called VERIFIED (coverage 80.93%), only 88.94% passed the hidden tests. By arithmetic on those two file values (not a measurement), about 8.95% of all problems were confident-wrong under v3.

2 Corrections since v3.3.1

An independent audit checked 99 claims of the v3.3.1 paper against raw rows, generations, the ledger, logs and live external records (`docs/CLAIM_AUDIT.md`; summary in Appendix A). It found 4 WRONG and 3 OVERSTATED claims. Each one, and how it is fixed here:

- **A32 (wrong hash).** v3.3.1 quoted the sha256 of `papers/numbers_v4.json` (v3.3.0) as `e5488f6c...`. That value is the file as committed in `b5be32a`, before the spend repricing. The file at tag `v3.3.0` hashes to `fccb1695...5995`. I recomputed this with `git show v3.3.0:papers/numbers_v4.json | sha256sum`. Fixed in Section 11.
- **A33 (wrong hash).** The quoted v3.3.1 hash `952bfc77...` belonged to an uncommitted scratch regeneration. The file at tag `v3.3.1` and on HEAD hashes to `5f3c2a6e...a1c0` (recomputed). Fixed.
- **A28 (stale spend).** v3.3.1 reported 5 ledger lines totalling \$1.54. That was true when v3.3.1 was released, but later TPU runs added lines. Section 10 now gives the full ledger; for revision 3.4.0 it held 8 lines, \$2.5289 in total, and for revision 3.5.0 it holds 11 lines, \$4.9136.
- **C12 (bibliography).** The `gwayav3` title now matches the Zenodo record: “GWAYA v3.1: Fail-Closed Gate & Real Toolchain Verification for LLM Code Generation”.
- **B15 (H13 “by construction”, overstated).** v3.3.1 said the ledger refuses any stage that would exceed its cap. The ledger file enforces nothing. Corrected wording (Section 7): the GPU stage launcher is designed to refuse a stage that would push the ledger over its cap, so H13 was expected to hold largely by construction (red-team prior 0.75). The TPU runs of v3.3.1 were launched by separate scripts with a run-time limit but no ledger check, so the low total reflects low usage. In either case it is not evidence about the GwenLaya system.
- **B12 (CPU-vs-TPU check, overstated).** v3.3.1 called TPU run #3 a consistency check “across hardware and quantisation”. In fact the engine, the code system prompt and the prefill/stop also changed. The run #3 reading now uses the audit’s wording (Section 9). The new same-protocol comparison A7 (Section 6.5) replaces it as the CPU-vs-TPU comparison.
- **A README claim (C17, overstated)** was fixed in the README in commit `39a5a0a`. It is not part of this paper.

Audit notes (VERIFIED_WITH_NOTE or NOT_VERIFIABLE) that are addressed in the text:

- Rust assert counting (B3): the harness counts `assert!`, `assert_eq!` and `assert_ne!` calls.
- ECE binning (A5): ECE is computed with 15 equal-mass bins.
- Run #2 (D12, B8): the run #2 *scores* are not used; only its generation count is cited. The reason is the run #2 gate self-test (`python_ok False`), not D29, which describes the re-scoring machine.
- TPU-host Python failure in run #3 (B9): the cause is not recorded, and the gate self-test on the same host passed.
- Registration timing (D11): the hypotheses were registered before any GwenLaya evaluation data were generated (registration first committed in `0b4f104`, 2026-10-07 23:33). This was after the TPU smoke tests, which produced no study data.
- “72 minutes” for L2 (A30) is the sum of sequential per-call times, not a start-to-end timestamp.
- The run #3 timers (A24) are self-reported by the run.

- LoRA (B14): the absence of a TPU LoRA path is limited to the stacks inspected. vLLM-TPU LoRA was tried only on Qwen2.5.
- Laya (C14): we did not verify that the `convaiinnovations/laya` model is an encoder, and the text now calls it a model.
- MultiPL-E (C2): volume, number, pages and DOI were added to the bibliography.
- Missing scripts (B4, B10): the scripts behind the v3.3.1 TPU and Rust-validation numbers are now committed (`scripts/validation/rust_reference.py`, `scripts/validation/rescore_rust.py`, `scripts/tpu/descriptive_run3/`, commit 39a5a0a). I did not re-run them for this revision.

2.1 Corrections since 3.4.0 (revision 3.5.0)

- **The math-gate gap statement is superseded.** Revision 3.4.0 said that the math gate was not implemented and had coverage 0 (abstract, Sections 6.4 and 12, D37, the hypothesis table and the next steps). The registered program-of-thought gate is now implemented (`gwaya/domains/math_gate.py`, addendum A10) and its results are in Section 6.6. The statement was true of the 3.4.0 run. It is no longer true of this revision. D37 stays in the record, with D43 added.
- **3.4.0 numbers that depended on the empty math gate are superseded, not deleted.** These are the math and pooled rows of Tables 5 and 6, the 757 cascade abstentions, the pooled selective figures (coverage 0.507, confident-wrong rate 0.103), the cascade-to-4B cost ratios 1.357 and 1.243 (pooled) and 1.514 (math), the pooled accuracy 0.604 and the pooled McNemar counts 40 and 44. Where they still appear in Section 6.4 they are labelled as 3.4.0 values. The current values are in Tables 8 and 9. The Python and Rust figures of those tables did not change. The pooled A4 difference (-0.063 in 3.4.0, diluted by 500 empty math tasks) is replaced by -0.090 .
- **Spend.** 3.4.0 reported 8 ledger lines and \$2.5289. The ledger now has 11 lines and its recomputed sum is \$4.9136 (Section 10).
- **Cost of gated math arms.** The 3.4.0 cascade did not charge anything for a math gate because none ran. From A10 on, every gated arm is charged for the program that its gate executes, for each tier it consults.

2.2 Corrections after a re-audit of revision 3.4.0

A second independent audit checked 144 claims of the 3.4.0 draft (`docs/CLAIM_AUDIT_340.md`). All E-set numbers reproduced, but it found 4 WRONG and 8 OVERSTATED rows (5 distinct issues) in explanations, definitions and timing. All are fixed in this text:

- **Pooled A4 (E47).** The old pooled -0.042 used one log-prob cut-off across all domains, so the baseline answered 280 math tasks that the gate never answers. It is replaced by a pooled difference with coverage matched within each domain (-0.063 , Section 6.4).
- **“Threshold tuned on E favours the baseline” (AB11, E45).** This was unsupported and is removed. The baseline answers the k most confident tasks, with k equal to the gate’s answered count in the same domain. No outcome labels are used to choose k , and ties are broken pessimistically. We claim no direction of bias.
- **Commit attribution of the scoring runs (E25).** `results.json` records the HEAD at the *end* of a run. The scoring runs started before the commits recorded there (Section 6.3).
- **“Test H1/H3 as registered on these generations” (NS1).** This is impossible: the registration freezes the calibrator and thresholds before E is generated (Section 7).

- **Plan timing (AB4, E3).** The 2B/4B scoring started 60s *before* the plan commit.
- **Engine concurrency (E14).** The engine logs report KV capacity for at most 54 (2B) and 11 (4B) concurrent requests, not the configured 96 and 32. The fatal 4B error at 96 is now quoted correctly.
- **Math gate (E42; superseded in 3.5.0 by Section 6.6).** The zero math coverage was an implementation gap in our checker, not a property of math tasks.
- **Smaller items.**
 - A throughput typo: 4,865, not 4,866, tok/s.
 - Cascade abstentions are now reported.
 - The cluster bootstrap is effectively item-level, and this is now stated.
 - The Python cascade interval is called borderline.
 - Truncated answers are described as having “no complete `\boxed{}`”.
 - Truncation analysis is in the plan, not post hoc.
 - The hash files were split by root.
 - A warm-up-excluded cost (S2) is now reported next to the planned cost.

2.3 Corrections made in v3.3.1 (history)

Revision v3.3.0 contained a wrong Rust result caused by our own harness. v3.3.1 corrected it and recorded four deviations (D27–D30). All were found and fixed after the night generations existed. No registered test had run, so none was written with a test result in view.

- **D27, Rust harness (v3.3.0 Rust result withdrawn).** Inside the bubblewrap sandbox, `rustc`’s linker `cc` resolves through `/etc/alternatives`, which the sandbox did not mount. As a result, *every* Rust program failed to link, and the v3.3.0 Rust result (0/26) was a harness artefact.
 - **Forgery.** The harness also wrote its pass nonce into `main.rs`, so a candidate could forge a pass. We reproduced this and closed it: the nonce now goes on `stdin`, `compile` and `run` use separate sandboxes, the `run` sandbox has no `/proc`, and `unsafe`, `link_section` and `asm` are rejected.
 - **Assert counting.** The harness counts `assert!`, `assert_eq!` and `assert_ne!` calls.
 - **Validation.** Hand-written reference solutions pass for 25 of the 26 night Rust tasks (25/25), and 9 deliberately wrong solutions all fail (9/9) (`results/gwenlaya_v4/rust_harness_validation.json`).
 - **Re-score.** The unchanged night generations now give Rust 6/26.
- **D28, math prompt path and scoring.** The night math generations went through the code path, so they remain invalid. For future runs the code was fixed in three ways: a prose math prompt that ends in `\boxed{}`, non-thinking mode for Qwen3.x, and boxed-only scoring as the default. Under boxed-only, all 26 night math generations are UNDECIDED.
- **D29, Python harness environment.** A `uv`-managed interpreter could not start inside bubblewrap on the re-scoring machine. This was fixed by recreating the interpreter symlinks. The Python result is unchanged (11/28).
- **D30, TPU run #3 (descriptive).** A third TPU run generated all 80 night tasks for three models. Python scoring on the TPU host failed for every item, for a reason that was not recorded. All TPU generations were therefore re-scored on our machine.

3 What GwenLaya is and what was registered

The full registration is `docs/GWENLAYA_PREREGISTRATION.md` (revision 2, design freeze; sha256 of each revision in `docs/PREREG_FREEZE.txt`). In brief:

- **System (GL).** The design has the following steps:
 1. Generate with a small Qwen3.5 tier.
 2. Run a *gate-visible* executed check that, by design, never sees the scoring tests. In the night task files this was violated for HumanEval+, whose gate payloads contained the whole hidden suite. No gate ran, and the files were rebuilt (D25, Section 5.1).
 3. Score the candidate with a calibrated confidence head (a LoRA head on the `convaiinnovations/laya` model).
 4. Answer VERIFIED or LIKELY_CORRECT(p), abstain, or escalate to a larger tier.

The calibration label is the outcome on the *hidden* checks, so VERIFIED is never treated as ground truth.

- **Eval pool E.** HumanEval+ and MBPP+ [11] (Python, 542 items), MultiPL-E Rust [3] (510), MATH-500 [7, 10] (500), and miniF2F Lean 4 (244, the registered size; outside the pooled analysis).
- **Arms.** B1 smallest tier alone; B2 largest tier alone; B3 log-probability selective prediction [4] with its threshold set on a separate split C; B4 self-consistency; B5 largest tier with the v3-style gate only; B6 Laya zero-shot; LR; and a shuffled-label negative control SH.
- **Primary family** (Holm [8], $m = 2$; a primary that does not run enters with $p = 1$, so the other is tested at 0.025).
 - H1: CWR(GL)–CWR(B3) at matched coverage, by exact one-sided McNemar [12] plus a cluster-stratified paired bootstrap; MEI 3.0 points.
 - H3: GPU-seconds per correct answered item, GL vs B5; MEI 25% reduction, with answered accuracy non-inferior by 2.0 points.
- **Secondary:** calibration (H4 ECE [6], H5 Laya vs LR AUROC, H6 split-conformal selective error [1]), quantization (H8), training (H9), and systems (H13 budget, H14 endpoint latency, H15 preemption). **Exploratory:** H2, H7, H10–H12.

The hypothesis wording and MEIs are unchanged. VERIFIED keeps its registered meaning: all gate-visible checks passed. The outcome on the hidden checks is written PASS_HIDDEN / FAIL_HIDDEN / UNDECIDED.

4 The night run and its deviations

The night of 2026-10-07/08 was meant to run the primaries cheaply. Two constraints shaped it:

- the total GCP budget (\$50 hard, \$45 working ceiling, \$40 planned);
- the project’s only GPU quota unit (GPUS_ALL_REGIONS limit 1), which a VM outside this project held all night. We did not touch that VM.

All night generation therefore ran on a shared 8-core CPU, and the scope was cut. Every deviation is in `docs/DEVIATIONS.md`. The night deviations are summarised in Table 1, and those of revisions 3.4.0 and 3.5.0 in Tables 14 and 15 (Section 6). When each group was written:

- D1–D20: before generation.
- D21: during L2.

- D22–D23: after the L2 generations existed.
- D24–D26: after the external review of the first draft.
- D27–D30: after release v3.3.0.
- D31–D41: during and after the full-E TPU baselines of revision 3.4.0.
- D43–D47: revision 3.5.0, all after the E baselines had been seen (Table 15). The D42 note concerns the burned E set (Section 7).

No registered test has run, so none of these was written with a registered test result in view. Some of D31–D41 and all of D43–D47 were written after exploratory results had been seen, and they say so.

Table 1: Deviations from the registration for the night run (from `docs/DEVIATIONS.md`). Reason codes: B = budget, G = no GPU slot, T = CPU wall clock, I = implementation or defect.

#	Registered	Night run	Why
D1	E = 1,552 items on L4	E-night: seeded prefix, 80 items (python 28, rust 26, math 26)	G, T
D2	tiers 2B/4B/9B/27B	2B and 4B only (B2/B3/B5 on 4B)	G, T
D3	H3 cost in L4 GPU-seconds	CPU-seconds on this machine	G
D4	separate calibration set C	5-fold cross-fitting on E-night by source cluster	T; C was not label-matched
D5	Laya LoRA r=16 heads	LoRA only if CPU timing fits, else frozen-embedding probe	T
D6	llama.cpp server	llama.cpp b11476 CPU; Ollama not used (no logprobs)	I
D7	Lean 4 (E, C, H11)	dropped tonight	T, B
D8	SFT, DPO, S4b–S5	deferred; H9 deferred, H10–H12 dropped	G, B
D9	B4 self-consistency, second seed	dropped	B, T
D10	GSM8K, AIME25, LiveCodeBench, MBPP anchor, 0.8B	dropped	B, T
D11	H8 on full E	300-item 9B subsample only if GPU; no TOST	G
D12	endpoint, 4–6 cold samples	4 samples, only after G1	G, B
D14–D15	<code>-t 8</code> ; 256-token tok/s probe	<code>-t 4</code> ; 64 tokens	T (foreign CPU load)
D16	–	fix: multi-letter words in math checker return undecided	I
D17	reference passes its own checks	Rust: typecheck only (no reference solution)	I
D18	decontaminate T, C	all 170 MBPP-sanitized calib items removed; no T/C	I
D19	n from measured throughput	n from measured tok/s and an <i>assumed</i> 196.6 tokens/item	T
D20	prompt text not fixed	gate-visible tests and prompts frozen in the task file	I
D21	–	logprobs and per-call CPU-seconds captured	I
D22	<code>random.Random(0)</code> bootstrap; primary Holm $m = 2$	numpy <code>default_rng(0)</code> , 10,000 resamples; Holm over the primaries that ran, reverted in D26	I
D23	–	scored the 80 L2 generations; single-arm descriptives added	I
D24	math: boxed answer only	scorer fell back to the last number; registered boxed-only rescoring reported too	I (review)
D25	gate never sees the scoring checks	HumanEval+ gate payloads held the full hidden suite; rebuilt, 5 primary items excluded; no gate ran	I (review)
D26	Holm $m = 2$	restored; per-domain descriptives, Python headline	I (post-L2)
D27	Rust: compile and run all hidden asserts	sandbox could not link any Rust program; fixed, nonce forgery closed; validated 25/25 correct, 9/9 wrong	I (post-v3.3.0)
D28	math: boxed answer, math prompt	night math went through the code prompt path (still invalid); prompt and boxed-only default fixed	I (post-v3.3.0)
D29	Python checks in bwrap	uv-managed interpreter could not start in the sandbox; fixed; Python unchanged	I (post-v3.3.0)
D30	–	unregistered descriptive TPU run #3 (3 models, 80 tasks); all rows re-scored locally	I, G (post-v3.3.0)

Benchmarks reduced. To stay within budget and wall clock, the night ran no benchmark beyond the three primary domains, on 80 of their 1,541 included items, with two tiers instead of four. Lean 4, GSM8K, AIME25, LiveCodeBench, the MBPP continuity anchor, the 0.8B tier, self-consistency and the second seed were dropped (D7–D10).

Power was known to be insufficient before generation. The freeze addendum computed the power of H1’s exact McNemar test at $n = 80$ before any item was generated: 0.052 at the 3-point MEI and 0.261 at a 6-point effect ($\alpha = 0.025$). These values depend on the assumed discordance rates.

What actually ran at night.

- L0 (environment, throughput) and L1 (manifest, freeze addendum, decontamination).
- L2: 2B generation on all 80 E-night items, about 72 minutes as the sum of sequential call times.
- Scoring of those 80 generations.

L3 (4B generation) was held because the L2 review found a blocking defect. G1 (GPU) and E1 (endpoint) never ran, because the GPU slot never freed.

5 Night results (CPU, 80 items, descriptive)

The scored arm **base** is Qwen3.5-2B q4_K_M on llama.cpp [5], greedy, thinking off, **max_new_tokens** 1024, with no gate, no calibrator and no escalation. It is not one of the registered arms. Confidence is the raw $\exp(\text{mean token log-prob})$, uncalibrated and not cross-fitted. Intervals are 95% cluster-stratified bootstrap percentile intervals (10,000 resamples, seed 0), and ECE uses 15 equal-mass bins. Every number below is read from `papers/numbers_v4.json`, written by `scripts/analyze_study.py` from the re-scored rows `results/gwenlaya_v4/night_L2fix/rows.jsonl`. Its tables are reproduced unedited in Appendix B.

Outcome labels. The per-row outcome here is the result of the *hidden* checks (`rows.jsonl` field **score**: VERIFIED, FAILED or UNVERIFIED, i.e. PASS_HIDDEN, FAIL_HIDDEN, UNDECIDED). It is not the registered VERIFIED. The **gate** field is null in all 80 night rows.

Why Python is the headline. Python ($n = 28$) was chosen as the headline in D26, before the Rust harness was fixed, and we keep that choice. Rust ($n = 26$) is a second descriptive domain. The math and pooled numbers are sensitivity rows only, because the night math generations are invalid.

Table 2: Night CPU arm **base**, raw confidence, descriptive (from `papers/numbers_v4.json`). Pass = PASS_HIDDEN. 95% cluster bootstrap CI in brackets. ECE: 15 equal-mass bins. Cost = server CPU-seconds per passing item (D3, D21). TBD = undefined (no passing item).

Subset	n	Pass rate	Mean conf.	ECE	AUROC	Cost / pass
Python (headline)	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
Rust (descriptive, D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
<i>Sensitivity only, not valid estimates</i>						
Math (invalid), boxed-only	26	0.000	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD	TBD
Pooled (boxed-only math)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

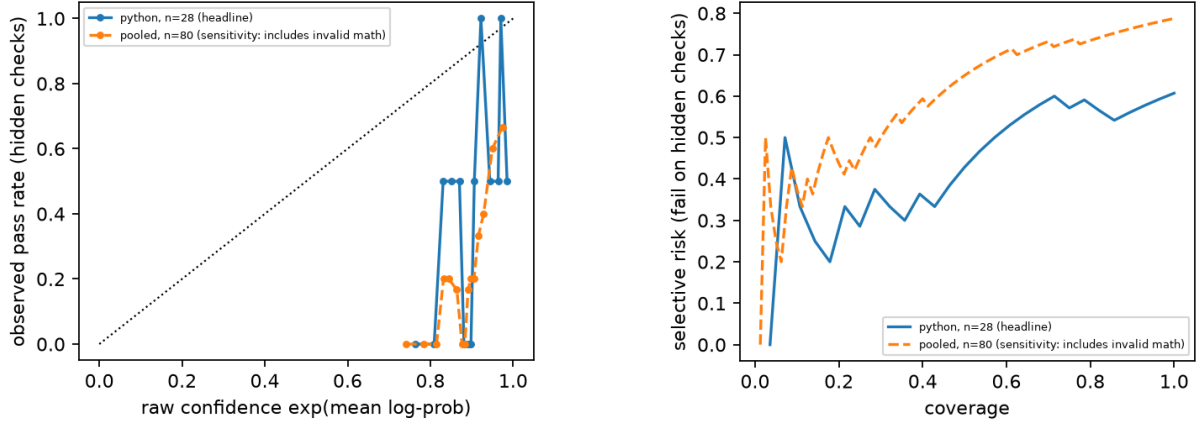


Figure 1: Night CPU arm **base**, raw confidence (v3.3.1 re-scored rows). Solid: Python, $n = 28$ (headline). Dashed: all 80 items. This is a sensitivity curve only, because it contains the invalid math items. Left: reliability diagram (equal-mass bins). Right: risk-coverage curve, where risk is the share failing the hidden checks. Descriptive only; not cross-fitted.

Python. 11 of 28 items passed the hidden checks and 17 failed (pass rate 0.393, CI 0.214–0.571). The mean raw confidence was 0.896, so the score is badly calibrated (ECE 0.519; Brier 0.476, CI 0.334–0.617). The AUROC interval (0.483–0.917) includes 0.5. The confident-wrong rate, counting every answered item that did not pass, was 0.607 (CI 0.429–0.786). The cost per passing item was 330 server CPU-seconds.

Rust. 6 of 26 items passed (0.231, CI 0.077–0.385). Mean raw confidence was 0.894 (ECE 0.675). The AUROC point estimate was 0.875 (CI 0.709–0.990). There were 6 passing items, one seed and an uncalibrated score, so we do not read this AUROC as evidence of good ranking. The harness was validated after these generations existed, on hand-written solutions, not on an independent reference set.

Math (not interpretable). Under the registered boxed-only rule, all 26 night math items are UNDECIDED. None of the generations contains `\boxed`, because they were produced through the code prompt path.

5.1 Defects found in the night data

1. **Math prompt path (blocking).** All 26 math completions were produced with the Python code system prompt and a `python` prefill, because the prompt builder fell back to the `python` tag. The fix is in the code (D28). The 26 night items have not been regenerated on CPU. The math generations of Section 6 use the fixed prompt.
2. **Unregistered math answer extraction (D24).** The v3.3.0 scorer fell back to the last number in the text. Since v3.3.1 the registered boxed-only rule is the default.
3. **Rust harness (D27).** No Rust program could link in the sandbox, and the pass nonce could be forged. Both are fixed and validated (25/25, 9/9).
4. **Python harness environment (D29).** Fixed; the Python result is unchanged.
5. **Gate not recorded at night.** The `gate` field is null in every night row.
6. **Hidden tests in the HumanEval+ gate payload (D25).** 11/11 E-night and 161/161 primary HumanEval+ gate payloads contained the hidden `inputs/results` lists. No reported number changes, because the L2 run executed no gate. The task files were rebuilt

with the prompts and hidden checks unchanged. E-night has 80 items; primary E has 1,536 items after 5 fail-closed exclusions. All E-set runs in Section 6 use the rebuilt `.d25` primary file.

What “passed” means in this paper. An item is `PASS_HIDDEN` only if it passed the named hidden checks:

- Python: the full EvalPlus test suite, run in a bubblewrap sandbox.
- Rust: compilation with `rustc`, then all `assert! / assert_eq! / assert_ne!` calls of the MultiPL-E test, run in a separate sandbox.
- Math: sympy/numeric equivalence between the gold answer and the last `\boxed{}` answer. If there is no `\boxed`, the item is `UNDECIDED` and is not counted as correct.

It means nothing more.

6 Exploratory full-E baselines on one TPU v5e chip (not registered arms)

This section is the main addition of revisions 3.4.0 and 3.5.0. Sections 6.1 to 6.5 report the 3.4.0 analyses (with the 3.5.0 supersessions marked); Sections 6.6, 6.7 and 6.8 are new in 3.5.0. None of it is a pre-registered test. H1 compares GL with B3 and H3 compares GL with B5. GL needs the Laya router and calibrator, and B3 needs a threshold set on split C. Neither exists.

6.1 Analysis plan and what is post hoc

The analyses A1–A9 are defined in `docs/ANALYSIS_PLAN_E_TPU.md`, committed in `8ee9da3` at 2026-10-08 22:22:55. The original text hashes to `1f2fa2c1...0d46` (recomputed from `git show 8ee9da3:docs/ANALYSIS_PLAN_E_TPU.md`).

Timing, stated plainly.

- All generation was complete before the plan was committed.
- The 2B/4B scoring run started at 22:21:55, *60 seconds before* the plan commit, and finished at 22:36:38 (`results.json wall_clock`). Its log prints only progress counts, not outcomes.
- The scored rows of the Coder anchor were complete at 22:22:45, *10 seconds before* the plan commit. The Coder enters only A1, A2 and A9.
- To the best of our records, no scored E outcome was inspected before the commit. However, the descriptive 80-task runs (night CPU arm, TPU run #3) had already shown approximate pass rates for these models.
- The analysis script was committed in `5e10ad2` at 22:25.

Post-hoc amendments. Four dated amendments were made after the first analysis run and are post hoc (a fifth, after the re-audit of revision 3.4.0, is listed in Section 2.2):

1. A7 excludes math. The CPU night math came from the defective prompt, and the pooled comparison had been seen before this decision.
2. The A8 timeout audit became a correction procedure, the serial overlay.
3. The cascade cost result is reported as found, with no tuning of the gate or the tier pair afterwards.

4. A stability audit of passing rows and the fail-closed sensitivity S1 were added. This was decided before the passing-row audit finished.

Everything not listed in the plan is labelled post hoc where it appears. In particular, the warm-up-excluded cost S2 and the per-domain-matched pooled A4 were added after the re-audit. The truncation analysis, including the pass rate of truncated items, is in the plan (A9).

6.2 Generation

Setup.

- **Hardware and engine.** One on-demand TPU v5e chip (v5litepod-1, us-west4-a) and vLLM-TPU 0.31.0 [9], with the engine version taken from the logs.
- **Decoding.** bf16, greedy (temperature 0), `max_new_tokens` 1024, thinking off, with the log-probability of each sampled token recorded.
- **Prompts.** The prompts are the study’s raw Qwen protocol prompts. They are built by `OllamaGenerator.build_raw_prompt` with the stop sequences of `OllamaGenerator.stop_sequences`, which is the same code path as the llama.cpp CPU arm.
- **Code prompt unchanged.** The diff of `gwaya/generators.py` between the L2 commit 64441eb and HEAD leaves the code-domain system prompt, the “‘lang prefill and the “‘ stop unchanged. It changes only the math path (D28).
- **Driver scripts.** Generation used `scripts/tpu/gen_batch.py`, run by `deploy/tpu/run_e_gen.sh` on the VM, which sets `SKIP_JAX_PRECOMPILE=1`.
- **Tasks.** `tasks_E_primary.d25.jsonl`: 1,536 tasks (Python 529, Rust 507, math 500) in 1,079 source-problem clusters.

Models and concurrency. Qwen2.5-Coder-1.5B-Instruct (continuity anchor), Qwen3.5-2B and Qwen3.5-4B. vLLM loaded each model at revision `main`, so the exact weight commit is not pinned (a limitation).

Requested and effective concurrency. The configured `max_num_seqs` was 96 for the Coder and the 2B. For the hybrid Qwen3.5 models the engine logs show the KV cache being re-split. The 2B log says “re-split at 54 concurrent requests”, so its effective concurrency was at most 54.

The 4B at 96 failed to start with `ValueError: Cannot fit both KV pools under gpu_memory_utilization=0.92: the mamba pool needs 193 blocks (2 per request x 96 requests) ... (gen_qwen3.5-4b-bf16_s96.log)`. It then ran with `max_num_seqs` 32, and its log says “re-split at 11 concurrent requests”. The line “Mamba and attention pools together exceed the HBM budget” is an INFO message that also appears in the successful runs.

vLLM calls the recurrent-state pool “mamba”, although Qwen3.5 is a hybrid of full and linear attention layers. The 4B therefore ran at much lower effective concurrency than the 2B (at most 11 vs at most 54), which makes the 4B dearer per token (D33).

A first pass was set aside. A first E-generation VM (`gwenlaya-tpu-e-gen-210955`) produced 1,536 Coder and 1,536 2B generations. Both have `mean_logprob` null in every row, and the 4B failed at engine start. The maintainer reports that this pass requested `logprobs=0`, for which vLLM-TPU returns no log-probabilities. I did not see that pass’s code. The pass was not used, and everything was regenerated on a second VM (`gwenlaya-tpu-e-gen2-214445`) with `logprobs=1`. The committed `gen_batch.py` now stops after the first chunk if no log-probabilities come back (D40). All three raw files of the second pass have a log-probability in every row (`mean_logprob` non-null for $3 \times 1,536$ rows). Their sha256 values match

`results/gwenlaya_v4/e_tpu/SHA256SUMS_dataroot.txt` both on the data disk and in the data lake (verified).

Throughput (from the data-lake `gen_batch` logs, second pass). Each model’s tasks ran in 8 chunks of 192, with every chunk mixing all three domains. Chunk 1 was much slower in every case: 62s for the Coder, 144s for the 2B and 192s for the 4B. Chunks 2–8 took 9–10s, 21–23s and 50–55s, a throughput of 4,507–4,865, 2,864–3,120 and 1,085–1,128 tok/s. The slow first chunk matches lazy compilation; we did not instrument it. These are batched-throughput figures, not single-stream latency.

Totals. Summed over the 1,536 generations (`data/gens_summary.jsonl`):

- Completion tokens: 353,862 (Coder), 509,089 (2B) and 464,253 (4B).
- Attributed chip-seconds: 129.4, 296.7 and 561.0. These equal the sums of the chunk wall times, *including* the slow first chunk.

6.3 Scoring and verification

Scoring. Nothing was scored on the TPU host.

- `scripts/import_remote_gens.py` wrote the raw generations into the study’s hash-chained `gens.jsonl`, with the same keys and seeds.
- The unchanged `scripts/run_study.py -mode score` then scored them locally in the bubblewrap sandbox, with `GWAYA_ALLOW_UNISOLATED` unset.
- Chip-seconds are attributed by token share within each chunk.

The 2B/4B run replayed six arms over the same cached answers (`base`, `always_smallest`, `always_largest`, `gate_only`, `raw_confidence` and `gwenlaya`, the last being the static gate ladder). It used 5 parallel check workers (`-check-workers 5`, recorded in `results.json`). The Coder rows were scored with one check process; their `results.json` records no check-worker option. Both scoring runs ran from working trees with uncommitted changes. `results.json` records the HEAD at the *end* of each run (66e2313 for the Coder, 5e10ad2 for the 2B/4B, both `dirty: true`). That is not the code that started the run:

- The Coder scoring began at 22:05:17, before 66e2313 existed (22:08:27). Its arguments have no check-worker key, which matches the earlier command line.
- The 2B/4B scoring began at 22:21:55, before 5e10ad2 (22:25:43).

The code that ran was therefore 39a5a0a and 66e2313 respectively, plus uncommitted changes that I cannot identify. I re-ran `tests/test_import_remote_gens.py` and `tests/test_analyze_e_tpu.py` for this revision, and 22 tests passed.

Serial re-check of every non-passing code row (A8). Parallel checks can time out under CPU contention. Every non-passing Python/Rust `base` row was therefore re-checked one at a time: 644 (2B), 412 (4B) and 480 (Coder), 1,536 in all (`audit/recheck_*.json`). Three outcomes flipped from FAILED to VERIFIED:

- `py/MBPP/271`, for the 2B and for the 4B. This is a contention timeout.
- `rs/mbpp_130_max_occurrences`, for the 4B. Plan amendment 4 identifies this as a *non-deterministic* candidate: it iterates a Rust `HashMap` and takes the first maximum, so with tied counts the result depends on the per-run hash order. According to the amendment it passes about half the time. I did not re-measure that frequency.

How the corrections were applied. The correction is an *overlay* (`overlay/rows_overlay_tiers.jsonl`, 16 rows). It holds a serial re-score of all six arms for the two flipped tasks. The original hash-chained rows are untouched, and the overlay takes precedence in the analysis.

- `py/MBPP/271` becomes VERIFIED in all 8 affected rows (`serial_recheck_corrections`).
- The `rs/mbpp_130` rows re-scored as FAILED and stay FAILED in the headline.

A bug in the first overlay was found and fixed. Re-importing the corrected tasks alone re-attributed the chunk time over far fewer tokens, which inflated those tasks’ chip-seconds. The importer now keeps the original attributed seconds, and `test_subset_reimport_keeps_original_attributed_seconds` guards this.

Timeouts and passing-row stability.

- **Timeouts when run alone.** Five checks per model still time out when run alone (5 for the 2B, 5 for the 4B, 5 for the Coder, listed in the audit files). They remain FAILED.
- **Re-run of every passing row.** All 1,572 passing Python/Rust rows were re-run once: 392 (2B), 624 (4B) and 556 (Coder). None changed outcome (`audit/recheck_pass_*.json`). One re-run finds a 50/50 unstable candidate only half the time, so the correct reading is “no unstable passing row was observed”, not “none exist”.

Sensitivity S1 (fail-closed). S1 counts every item that produced two different outcomes in any run as not correct: 1 item for the 2B and 2 for the 4B. Pooled accuracy is then 0.4225 vs 0.4232 headline (2B), 0.6055 vs 0.6061 (4B) and 0.6029 vs 0.6035 (cascade). The largest difference in any domain is 0.0019 (4B Python, 0.6730 vs 0.6749). The headline and S1 agree well inside the confidence intervals.

Hash files. The hashes are kept in two files, so that each can be checked from its own root:

- `results/gwenlaya_v4/e_tpu/SHA256SUMS_repo.txt`, checked from the repository root;
- `SHA256SUMS_dataroot.txt` (the full generations, rows and raw files), checked from the data root.

I ran `sha256sum -c` on each from its root, and every entry matches.

6.4 Results

Tables 3–7 are `papers/tables_e_tpu.tex` exactly as written by `scripts/analyze_e_tpu.py` for revision 3.4.0 (input, not retyped). **In 3.5.0 their math and pooled gate and cascade entries (Tables 5 and 6) are superseded by Section 6.6**, because they were computed with an empty math gate. Every number in the text is read from `results/gwenlaya_v4/e_tpu/numbers_e_tpu.json` (key in parentheses where useful). All CIs are 95% source-problem cluster bootstrap percentile intervals (10,000 resamples, `numpy default_rng(0)`). Within each domain every cluster holds exactly one item (checked on the task file), so the bootstrap is in effect item-level, stratified by domain. Python and Rust problems from the same source are resampled independently. According to the re-audit’s own check, resampling them together would widen pooled CIs by at most 0.004. “Correct” means PASS_HIDDEN. FAILED and UNVERIFIED both count as not correct.

A1, pass rates (Table 3). On all 1,536 tasks the 4B passes 0.606 of the hidden checks, the 2B 0.423 and the Coder 0.482. On Python the Coder (0.647) is close to the 4B (0.675). These are single greedy samples on public benchmarks the models have probably seen. They are not clean capability estimates.

Table 3: Pass rate on the hidden checks, full E set, greedy, one sample (95% cluster bootstrap CI). TPU v5e bf16.

Model	Python (529)	Rust (507)	Math (500)	Pooled (1536)
qwen3.5-2b-bf16	0.452 [0.408, 0.493]	0.304 [0.264, 0.345]	0.514 [0.470, 0.558]	0.423 [0.398, 0.447]
qwen3.5-4b-bf16	0.675 [0.635, 0.715]	0.529 [0.485, 0.574]	0.612 [0.568, 0.654]	0.606 [0.582, 0.631]
qwen2.5-coder-1.5b-bf16	0.647 [0.605, 0.686]	0.422 [0.381, 0.465]	0.368 [0.326, 0.410]	0.482 [0.457, 0.507]

Table 4: Raw confidence $\exp(\overline{\log p})$ as a correctness score (ECE: 15 equal-mass bins).

Model	Domain	mean conf.	ECE	AUROC	AURC
qwen3.5-2b-bf16	python	0.898 [0.893, 0.903]	0.446 [0.405, 0.488]	0.649 [0.601, 0.694]	0.455 [0.398, 0.515]
qwen3.5-2b-bf16	rust	0.902 [0.898, 0.906]	0.598 [0.559, 0.636]	0.772 [0.728, 0.813]	0.512 [0.453, 0.573]
qwen3.5-2b-bf16	math	0.902 [0.898, 0.905]	0.388 [0.345, 0.430]	0.741 [0.697, 0.784]	0.332 [0.276, 0.389]
qwen3.5-4b-bf16	python	0.927 [0.923, 0.931]	0.252 [0.215, 0.291]	0.681 [0.631, 0.728]	0.209 [0.169, 0.254]
qwen3.5-4b-bf16	rust	0.930 [0.927, 0.933]	0.402 [0.359, 0.443]	0.775 [0.733, 0.815]	0.279 [0.230, 0.331]
qwen3.5-4b-bf16	math	0.938 [0.935, 0.940]	0.326 [0.284, 0.369]	0.697 [0.646, 0.746]	0.285 [0.229, 0.341]
qwen2.5-coder-1.5b-bf16	python	0.915 [0.911, 0.919]	0.268 [0.230, 0.309]	0.722 [0.674, 0.766]	0.215 [0.173, 0.261]
qwen2.5-coder-1.5b-bf16	rust	0.934 [0.931, 0.937]	0.512 [0.470, 0.554]	0.773 [0.731, 0.812]	0.366 [0.313, 0.421]
qwen2.5-coder-1.5b-bf16	math	0.932 [0.930, 0.935]	0.564 [0.523, 0.606]	0.615 [0.565, 0.665]	0.571 [0.509, 0.630]

A9, truncation. Math answers are often cut at 1,024 tokens. The share of math generations with `finish_reason = length` is 0.400 (2B), 0.320 (4B) and 0.152 (Coder) (`A9.trunc.*.math`). The code shares are at most 0.053 (2B Python).

From the rows, no truncated generation passed in any model or domain. Five truncated math answers contain the string `\boxed` (1, 1 and 3 for the Coder, 2B and 4B; read from the raw files), and four of these are unclosed. The truncated math answers therefore have no complete `\boxed{}` and are UNVERIFIED, except one 2B item that was FAILED. The math pass rates are therefore lower bounds under this token budget, and the math comparison between models partly measures verbosity.

A2, raw confidence (Table 4). $\exp(\text{mean log-prob})$ sits near 0.9 in every cell, from 0.898 (2B Python) to 0.938 (4B math), whatever the accuracy. On the 4B the mean confidence is 0.927 / 0.930 / 0.938 (Python / Rust / math), against pass rates of 0.675 / 0.529 / 0.612. ECE ranges from 0.252 (4B Python) to 0.598 (2B Rust). The score is not usable as a probability without calibration.

Its ranking is weak to moderate. AUROC lies between 0.615 (Coder math) and 0.775 (4B Rust), and every CI excludes 0.5. This does not test H4 or H5, which concern the Laya head.

A3, gate-only on the 4B (Table 5). The executed gate runs the gate-visible test of each task. On Python it answers 0.813 of the tasks, and 0.830 of those pass the hidden checks. On Rust the figures are 0.596 and 0.887. Over all tasks, the confident-wrong rate falls from 0.325 to 0.138 (Python) and from 0.471 to 0.067 (Rust), compared with answering everything. That reduction comes from abstaining on 19 % and 40 % of the tasks.

On math the 3.4.0 gate answered nothing (coverage 0, precision undefined). That was an *implementation gap* in our checker, not a property of math tasks. The checker executed only the task’s `gate_payload`, which is empty for all 500 math rows, so every math gate verdict was UNVERIFIED. The registered math gate is a sandboxed program-of-thought re-execution that reproduces the boxed answer (registration Section 1). It needs no task payload. **It is implemented in revision 3.5.0 and its results replace the empty row (Section 6.6).**

Both code-domain precisions are below the 0.95 point that the registration uses to flag a gate weakness (H7). This is a descriptive analogue on unregistered hardware and tiers, not an H7 result.

Table 5: Gate-only on qwen3.5-4b-bf16 (answer iff the executed gate verifies) versus answering everything, and versus a log-probability baseline that answers the same number of tasks (the k most confident, k = the gate’s answered count in that domain; no labels are used to choose k).

Domain	coverage	precision	CWR gate	CWR answer-all	Δ vs matched log-prob
python	0.813 [0.779, 0.847]	0.830 [0.795, 0.864]	0.138 [0.110, 0.168]	0.325 [0.285, 0.365]	-0.076 [-0.098, -0.057]
rust	0.596 [0.552, 0.639]	0.887 [0.851, 0.922]	0.067 [0.045, 0.091]	0.471 [0.426, 0.515]	-0.112 [-0.134, -0.093]
math	0.000 [0.000, 0.000]	TBD	0.000 [0.000, 0.000]	0.388 [0.346, 0.432]	0.000 [0.000, 0.000]
all (per-domain matched)	—	—	0.070 [0.058, 0.082]	—	-0.063 [-0.074, -0.054]

Table 6: Exploratory gate cascade (qwen3.5-2b-bf16 $\rightarrow$ gate $\rightarrow$ qwen3.5-4b-bf16) versus always-smallest and always-largest. Cost = accelerator chip-seconds per task (throughput at each tier’s effective generation concurrency, which differed between tiers; not latency).

Domain	acc. small	acc. large	acc. cascade	cost ratio casc./large	escalated
python	0.452 [0.408, 0.493]	0.675 [0.635, 0.715]	0.648 [0.607, 0.688]	0.997 [0.925, 1.071]	0.408 [0.365, 0.450]
rust	0.304 [0.264, 0.345]	0.529 [0.485, 0.574]	0.548 [0.505, 0.594]	1.221 [1.153, 1.291]	0.633 [0.592, 0.675]
math	0.514 [0.470, 0.558]	0.612 [0.568, 0.654]	0.612 [0.568, 0.654]	1.514 [1.489, 1.538]	1.000 [1.000, 1.000]
pooled	0.423 [0.398, 0.447]	0.606 [0.582, 0.631]	0.604 [0.579, 0.628]	1.357 [1.329, 1.385]	0.675 [0.655, 0.695]

A4, gate vs a log-prob threshold at matched coverage (exploratory; most favourable result). At the gate’s own coverage, “answer the most confident tasks” gives a confident-wrong rate of 0.214 on Python and 0.180 on Rust. The gate gives 0.138 and 0.067. The differences are -0.076 (CI -0.098 to -0.057) and -0.112 (CI -0.134 to -0.093) (A4.delta_cwr_gate_minus_b3.qwen3.5-4b-bf16.*). Figure 2 shows the same comparison as risk-coverage curves.

Several caveats limit this result:

1. The gate executes a visible test, which is information the log-prob does not have. Part of the gap is simply “running a test helps”.
2. The baseline answers the k most confident tasks, with k equal to the gate’s answered count in the same domain (recomputed in each bootstrap resample). No outcome labels are used to choose k , and ties are broken pessimistically. k is matched on E rather than taken from a threshold frozen on a separate split. We do not claim a direction for any resulting bias.
3. The benchmarks are public and probably in pretraining.
4. There is a single greedy sample.
5. This is **not** H1. H1 compares GL (gate plus Laya) with B3, whose threshold is set on a separate split C, and requires the coverage difference to lie within ± 3 points.

In 3.4.0 the gate answered no math task, so the matched baseline answered $k = 0$ tasks, the math difference was 0 by construction, and the pooled difference with coverage matched within each domain (-0.063 , CI -0.074 to -0.054) was the Python and Rust differences diluted by 500 empty math tasks. Both are superseded by the 3.5.0 values with the math gate (math -0.084 , pooled -0.090 ; Table 8).

A5, the gate cascade (exploratory, not GL; Table 6, Figure 3). All pooled and math figures in this paragraph and the next bullet lists are 3.4.0 values, computed with an empty math gate and no program cost. They are superseded by Table 9 and Section 6.6. The Python and Rust figures are unchanged in 3.5.0. The cascade works as follows. The 2B answers. If the executed gate verifies, that answer is final; otherwise the task goes to the 4B and the 4B’s answer is final. This is GwenLaya without Laya. It is compared with always-smallest (B1) and always-largest (B2), not with the registered B5. Holm is taken over these two contrasts.

In 3.4.0 the cascade arm *abstained* when the 4B’s gate did not verify. This happened on 757 of 1,536 tasks: all 500 math tasks, 77 Python and 180 Rust. The “cascade accuracy” below is the

Table 7: Same raw protocol, night Python and Rust tasks (math excluded: the CPU math used a defective prompt): Qwen3.5-2B q4_K_M llama.cpp CPU versus bf16 vLLM TPU. Hardware, quantisation and engine all differ.

Domain	n	CPU pass	TPU pass	agree	McNemar p
python	28	11	10	27	1.000
rust	26	6	8	22	0.625
pooled	54	17	18	49	1.000

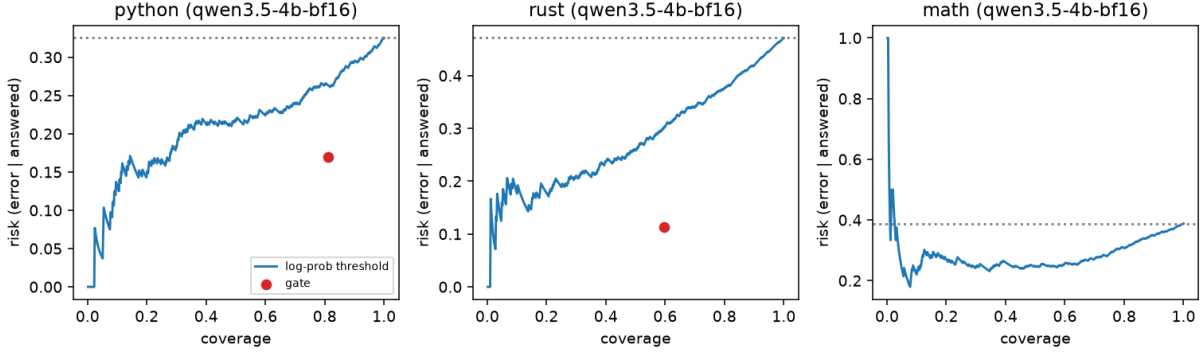


Figure 2: Qwen3.5-4B (bf16, TPU), full E set, exploratory. Blue: risk–coverage curve of a raw log-prob threshold. Here risk is the error rate *among answered tasks*, while the tables report CWR over *all* tasks. Red dot: the executed gate, whose risk is one minus its precision. Dotted line: error rate when every task is answered. The curve is traced on E by ranking tasks by confidence; no labels choose the cut-off. This is the 3.4.0 figure: on math our gate then verified nothing, so there is no red dot. Figure 4 replaces it with the math gate. The left end of each curve rests on very few tasks.

hidden pass rate of the final candidate, whether or not the arm answered. Its selective metrics are:

- pooled coverage 0.507 (CI 0.490–0.525) with confident-wrong rate 0.103 (CI 0.089–0.117);
- Python 0.854 and 0.206;
- Rust 0.645 and 0.097;
- math coverage 0 (A5.cascade_selective_*; superseded: 3.5.0 pooled 0.669, CWR 0.113, math 0.496 and 0.030).
- **No accuracy gain.** Pooled accuracy is 0.604 for the cascade and 0.606 for the 4B, a difference of -0.003 (CI -0.014 to 0.009). The paired exact McNemar test gives 40 tasks correct only under the cascade and 44 only under the 4B, $p = 0.744$ (Holm 0.744). On Python the cascade is slightly worse: -0.026 (CI -0.053 to -0.002). The upper bound is within Monte-Carlo noise of 0 (the re-audit obtained 0.000), so this is borderline. Gate-verified 2B answers that fail the hidden checks are never escalated. On Rust the difference is $+0.020$ (CI -0.004 to 0.043). Against the 2B alone, the cascade gains 0.180 pooled (295 vs 18 discordant, Holm $p = 2.0 \times 10^{-65}$). That gain comes from the 4B.
- **Not cheaper than always-largest, pooled.** The pooled ratio of cascade to 4B chip-seconds is 1.357 (CI 1.329–1.385) with the first, compilation-bearing chunk counted, as planned. By domain it is 0.997 (CI 0.925–1.071) for Python, 1.221 for Rust and 1.514 for math.

Excluding the first chunk of every model gives the post-hoc sensitivity S2 (S2.cost_ratio_cascade_over_large_nowarm.*): pooled 1.243 (CI 1.224–1.260), Python

0.900 (CI 0.834–0.965), Rust 1.090 and math 1.398. Pooled, the cascade is more expensive either way. For Python it breaks even with warm-up and is about 10% cheaper without it, at a 2.6-point accuracy cost. The cascade escalates 0.675 of tasks pooled: 0.408 on Python, 0.633 on Rust and every math task. It therefore pays for the 2B and then for the 4B on most tasks. The cost per correct answer is 0.821 chip-seconds for the cascade (CI 0.759–0.889) and 0.603 for the 4B (CI 0.563–0.646).

- **Ceiling (A6).** An oracle that picks a correct tier whenever one exists would reach 0.644 pooled (CI 0.620–0.668), against 0.606 for the 4B. By domain it reaches 0.707 / 0.574 / 0.648 (Python / Rust / math), against 0.675 / 0.529 / 0.612. Any router between *these two* tiers can gain at most these differences in accuracy on these tasks. The oracle is a bound only and never a comparator.

Cost caveats. The chip-seconds measure throughput at each model’s engine concurrency, not latency. They are not comparable to the registered L4 GPU-seconds. Three points matter here:

1. The 4B ran at an effective concurrency of at most 11, against at most 54 for the 2B (configured 32 and 96). This makes the 4B dearer per token, so the measured cascade cost penalty is *understated* relative to an equal-concurrency setup.
2. The slow first chunk is a larger share of the 2B’s total (144 of 297s) than of the 4B’s (192 of 561s). S2 above removes it.
3. The CPU time of the gate itself is not counted.

Cost is therefore configuration dependent. The ratios above are not robust estimates of any deployment cost, and we draw no cost conclusion beyond “not cheaper pooled, in this configuration”. The accuracy result does not depend on cost.



Figure 3: Exploratory, not a registered arm. Pass rate against accelerator chip-seconds per task (TPU v5e, throughput at the engine’s concurrency: configured 96 for the 2B and 32 for the 4B, effective at most 54 and 11 per the engine logs; first chunk included; not latency; not comparable to GPU-seconds) for always-2B, always-4B and the gate cascade (2B → executed gate → 4B). The axes do not start at zero. On math the cascade escalates every task, so it costs the 2B plus the 4B for the 4B’s accuracy.

6.5 A7: same-protocol CPU vs TPU on the night tasks

This comparison uses the 54 night Python and Rust tasks, which are contained in the E set with identical prompts (D25). It compares the night CPU arm (Qwen3.5-2B q4_K_M, llama.cpp, re-scored v3.3.1 rows) with the TPU 2B (bf16, vLLM). Both use the same raw prompts, prefill and stop sequences (Table 7; A7_night_cpu_vs_tpu).

- **Python.** 11 vs 10 passes; 27 of 28 items agree.

- **Rust.** 6 vs 8 passes; 22 of 26 agree; McNemar $p = 0.625$.
- **Pooled.** 17 vs 18 passes; 49 of 54 agree; $p = 1.0$.

Hardware, quantisation and engine all differ, while the prompt text, prefill and stop do not. This is a consistency check with small n , not an isolation of any one factor, and not an equivalence test.

Math is excluded (plan amendment 1, post hoc) because the CPU night math used the defective code prompt (D28). Including it had given a pooled p driven entirely by that defect.

6.6 A10: the program-of-thought math gate (revision 3.5.0; post hoc, exploratory)

What changed. The math-gap statement of 3.4.0 is superseded (Section 2.1). The registered design (registration Section 1) is now implemented in `gwaya/domains/math_gate.py` and specified in addendum A10 of `docs/ANALYSIS_PLAN_E_TPU.md`, which was written before any program was generated and after the E baselines had been seen. The tier that answered writes a short Python program that computes the answer and prints it. The program runs in the same bubblewrap sandbox as the code checks (no network, no host files, time and memory limits). The gate VERIFIES iff the program’s last output line is equivalent to the answer’s final `\boxed{}` value, with the same equivalence as the scorer. The gold answer is never an input. A missing boxed answer, a missing program, a crash, a timeout, unparseable output or a mismatch all give UNVERIFIED (a wrong program does not refute the answer). One greedy program per tier and math task, at most 1,024 new tokens, same TPU, engine and settings as the answers.

Cost. The program is a second accelerator generation. Every gated arm (gate-only and the cascade) is charged the program’s attributed chip-seconds for each tier it consults; ungated arms are not. The program run and this charge are why the math cascade is dearer than in 3.4.0.

Gate-only (Table 8, Figure 4). Numbers are in the table (A3.*, A4.* of `numbers_e_tpu_mathgate.json`). On the 4B the gate verifies 0.454 of the math answers and 0.943 of those pass the hidden check, so the confident-wrong rate over all math tasks is 0.026, against 0.388 when every answer is given. At the same coverage, “answer the most confident” gives a higher rate; the difference is -0.084 . Pooled over all 1,536 tasks with coverage matched within each domain, the gate’s rate is 0.078 against 0.169 for the baseline (-0.090 , CI -0.104 to -0.079 ; A4.*.domain_matched). The 2B’s math gate verifies fewer answers (0.288) at precision 0.951. Python and Rust are unchanged from 3.4.0.

Reading and caveats.

1. The program is written by the same tier that wrote the answer. A match is agreement between two outputs of one model, not independent ground truth. Precision 0.943 means that a share of verified math answers still fail the hidden check; its point estimate is below the registration’s 0.95 flag point and its CI (0.911–0.971) includes it. This is a descriptive analogue, not an H7 result.
2. Coverage 0.454 means that 0.546 of the 4B’s math answers stay UNVERIFIED for any of the reasons above. We did not break the cause down.
3. As in A4, the gate runs a program that the log-prob does not see, k is matched on E, the benchmarks are public, and there is a single greedy sample. This is not H1.

Cascade with the math gate (Table 9, Figure 5). The cascade is the 3.4.0 one (2B, executed gate, then the 4B; the arm abstains if the 4B’s gate does not verify), now with the math gate and the program charge. Python and Rust are as in 3.4.0.

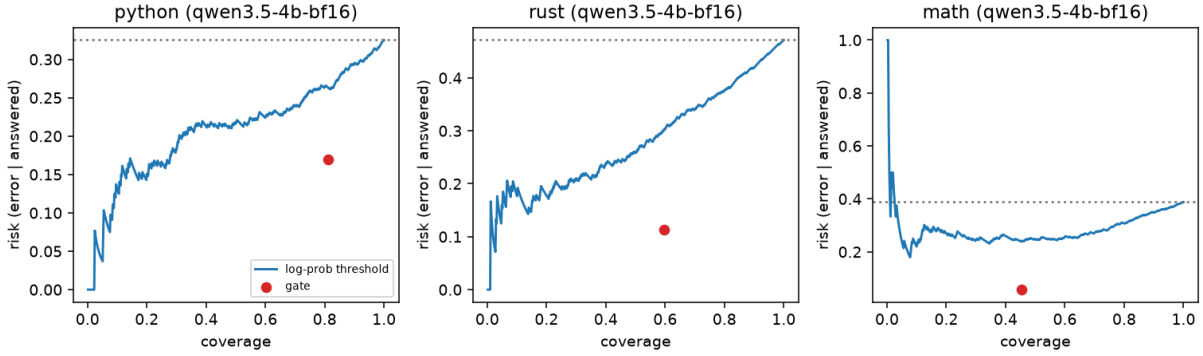


Figure 4: Qwen3.5-4B (bf16, TPU), full E set, exploratory; revision 3.5.0, with the program-of-thought math gate. Blue: risk–coverage curve of a raw log-prob threshold (risk = error rate *among answered tasks*). Red dot: the executed gate, whose risk is one minus its precision. Dotted line: error rate when every task is answered. Math now has a gate point. The left end of each curve rests on very few tasks.

Table 8: A10, gate-only with the program-of-thought math gate (answer iff the executed gate verifies). 4B: all domains; 2B: math only. CWR = confident-wrong rate over all tasks. Δ = gate CWR minus the per-domain coverage-matched log-prob baseline (A4). 95% cluster bootstrap CIs.

Tier	Domain	coverage	precision	CWR gate	CWR answer-all	Δ vs matched log-prob
4B	Math	0.454 [0.410, 0.498]	0.943 [0.911, 0.971]	0.026 [0.012, 0.040]	0.388 [0.346, 0.432]	−0.084 [−0.110, −0.062]
4B	Pooled	0.624 [0.602, 0.648]	0.875 [0.854, 0.895]	0.078 [0.065, 0.092]	0.394 [0.369, 0.418]	−0.090 [−0.104, −0.079]
4B	Python	0.813 [0.779, 0.847]	0.830 [0.795, 0.864]	0.138 [0.110, 0.168]	0.325 [0.285, 0.365]	−0.076 [−0.098, −0.057]
4B	Rust	0.596 [0.552, 0.639]	0.887 [0.851, 0.922]	0.067 [0.045, 0.091]	0.471 [0.426, 0.515]	−0.112 [−0.134, −0.093]
2B	Math	0.288 [0.250, 0.328]	0.951 [0.913, 0.985]	0.014 [0.004, 0.026]	—	—

- **No accuracy gain over always-4B.** Pooled accuracy is 0.605 for the cascade and 0.606 for the 4B (−0.001, CI −0.013 to 0.012). Exact McNemar gives 45 tasks correct only under the cascade and 46 only under the 4B, $p = 1.0$. Against the 2B alone the cascade gains 0.182 (293 against 13 discordant tasks, Holm $p = 8.2 \times 10^{-70}$; `paired_tests`); that gain comes from the 4B. On math the cascade gains +0.006 (CI −0.004 to 0.016) over the 4B.
- **Dearer than always-4B, with and without warm-up.** Pooled cost ratio 2.021 (CI 1.933–2.111) as planned and 2.023 (CI 1.912–2.134) without the first chunk (S2). On math 2.601 (CI 2.471–2.729) and 2.670 (CI 2.501–2.835): the cascade costs 1.784 chip-seconds per math task against 0.686 for the 4B, by construction of the charge: it generates a 2B answer and a 2B program for every task and escalates 0.712 of them to the 4B, which is then also charged for its own program. The cost per correct answer is 1.219 chip-seconds for the cascade pooled (CI 1.111–1.338) against 0.603 for the 4B (CI 0.563–0.646).
- **Selective view.** With the 4B gate as the last word the cascade answers 0.669 of all tasks pooled (CI 0.646–0.692) with confident-wrong rate 0.113 (CI 0.098–0.128); math 0.496 and 0.030; Python 0.854 and 0.206; Rust 0.645 and 0.097 (`A5.cascade_selective_*`). The “cascade accuracy” above counts the final candidate whether or not it was answered.
- **Ceiling (A6) unchanged.** The oracle over the two tiers reaches 0.644 pooled (CI 0.620–0.668) against 0.606 for the 4B.

The cost caveats of 3.4.0 (different effective concurrency, warm-up, gate CPU time not counted) apply, and the program charge adds one more: it assumes that every tier consulted on a math task also generates a program for the gate.

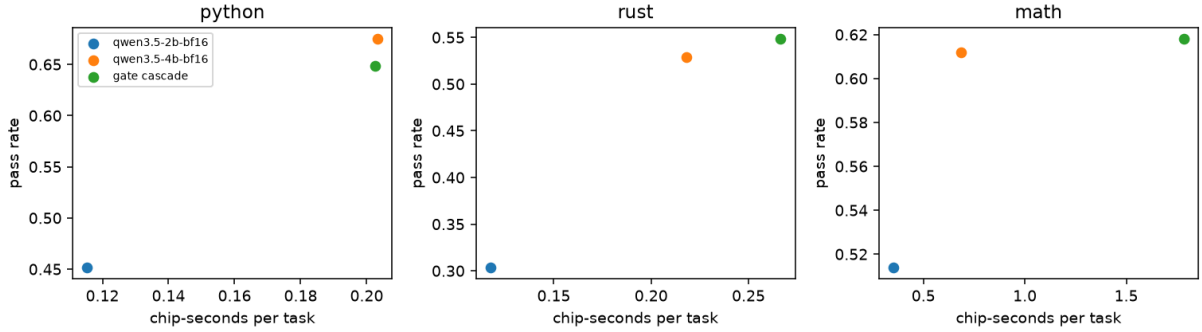


Figure 5: Exploratory, not a registered arm; revision 3.5.0 (with the math gate and the program charge). Pass rate against accelerator chip-seconds per task for always-2B, always-4B and the gate cascade (first chunk included; not latency; not comparable to GPU-seconds; concurrency differs between tiers). The axes do not start at zero and differ between panels.

Table 9: A10, gate cascade 2B $\rightarrow$ gate $\rightarrow$ 4B with the program-of-thought math gate, versus always-4B. Cost ratio = cascade chip-seconds / always-4B chip-seconds; programs are charged to the cascade for each tier consulted. “no warm-up” drops the first chunk of every model (S2, post hoc). Selective: coverage and CWR of the cascade’s final answer when it abstains if the 4B gate does not verify.

Domain	acc. 4B	acc. cascade	Δ acc. (casc.−4B)	cost ratio	cost ratio, no warm-up	escalated
Python	0.675	0.648	−0.026 [−0.053, −0.002]	0.997 [0.925, 1.071]	0.900 [0.834, 0.965]	0.408
Rust	0.529	0.548	+0.020 [−0.004, +0.043]	1.221 [1.153, 1.291]	1.090 [1.036, 1.140]	0.633
Math	0.612	0.618	+0.006 [−0.004, +0.016]	2.601 [2.471, 2.729]	2.670 [2.501, 2.835]	0.712
Pooled	0.606	0.605	−0.001 [−0.013, +0.012]	2.021 [1.933, 2.111]	2.023 [1.912, 2.134]	0.581

6.7 A11: cross-fitted calibrated abstention (revision 3.5.0; post hoc, exploratory)

Addendum A11 (written before any model was fitted) replaces a calibrator fitted on the non-existent split C with a fully pre-specified cross-fitted procedure run once on E. It is *not* the registered Laya calibrator and not H4, H5 or H6. Features come from stored outputs only (gate verdict, mean, minimum and lowest-10% token log-probability, log token count, truncation flag, domain); the gold answer and the hidden checks are never inputs. Folds are 5, by source problem (deterministic hash). M0 is the gate verdict alone, M1 uses the mean log-prob only, M2 all non-gate features and M3 all features, the last three as logistic regressions evaluated only on out-of-fold predictions (`scripts/analyze_abstention.py`; `abstention_a11.json`). The 4B is the headline tier and the 2B secondary. A11 was not run for the 9B.

Discrimination (Table 10). On the 4B, pooled AUROC is 0.851 (M0), 0.708 (M1) and 0.931 (M3); pooled AUC (lower is better) is 0.256 for M1 and 0.135 for M3. M3 beats M1 on AUROC by 0.223 (CI 0.197–0.249) and M0 by 0.079 (CI 0.064–0.095) (`disc.d_aucroc.*.pooled`); per domain the M3–M0 AUROC differences are 0.094 (Python), 0.029 (Rust) and 0.119 (math), all with CIs above zero. On the 2B pooled AUROC is 0.958 for M3 against 0.703 for M1. **M3 uses the gate verdict as a feature, so its advantage over the log-prob score is mostly calibration of the gate information and not independent evidence that a learned score is better than the gate.** Raw confidence was far worse calibrated (ECE 0.252–0.402 per domain on the 4B, Table 4) than the out-of-fold M3 (0.025–0.045).

Operating points (Table 11). With target selective risk $\alpha = 0.05$ the log-prob-only M1 never met the inner-fold rule (coverage 0 in every domain and pooled), while M3 answers 0.275 of tasks pooled (CI 0.254–0.298) at realised risk 0.052 (CI 0.031–0.074). At $\alpha = 0.10$ M3 answers 0.585

Table 10: A11 (4B), 5-fold cross-fitted out-of-fold scores. M0: gate verdict alone; M1: mean log-prob only; M3: logistic regression on all features (includes the gate verdict, so M3–M1 is a calibration gain, not independent evidence). Point [95% cluster bootstrap CI of fixed out-of-fold predictions]. ECE: 15 equal-mass bins; AUC: lower is better.

Domain	AUROC M0	AUROC M1	AUROC M3	AUC M1	AUC M3	ECE M3
Python	0.788 [0.750, 0.824]	0.681 [0.630, 0.728]	0.881 [0.846, 0.913]	0.211 [0.170, 0.255]	0.113 [0.087, 0.142]	0.045 [0.035, 0.077]
Rust	0.929 [0.906, 0.950]	0.772 [0.730, 0.812]	0.958 [0.939, 0.974]	0.280 [0.232, 0.332]	0.162 [0.131, 0.196]	0.025 [0.020, 0.052]
Math	0.816 [0.785, 0.846]	0.693 [0.642, 0.742]	0.936 [0.907, 0.960]	0.290 [0.233, 0.347]	0.138 [0.101, 0.179]	0.033 [0.025, 0.059]
Pooled	0.851 [0.833, 0.869]	0.708 [0.680, 0.734]	0.931 [0.916, 0.945]	0.256 [0.228, 0.285]	0.135 [0.116, 0.156]	0.021 [0.017, 0.039]

Table 11: A11 (4B), target-risk operating points: realised coverage and realised selective risk $P(\text{wrong} \mid \text{answered})$, pooled over the outer folds; threshold chosen inside each training portion for $\alpha \in \{0.05, 0.10\}$. M1 never met $\alpha = 0.05$ (coverage 0 in every domain, risk undefined). Gate: no threshold. Point [95% CI].

Domain	M3, $\alpha = 0.05$		M3, $\alpha = 0.10$		gate	
	coverage	risk	coverage	risk	coverage	risk
Python	0.174 [0.144, 0.208]	0.043 [0.010, 0.090]	0.622 [0.580, 0.664]	0.112 [0.080, 0.147]	0.813 [0.779, 0.847]	0.170 [0.136, 0.205]
Rust	0.264 [0.227, 0.304]	0.037 [0.008, 0.072]	0.533 [0.489, 0.576]	0.100 [0.066, 0.137]	0.596 [0.552, 0.639]	0.113 [0.078, 0.149]
Math	0.394 [0.352, 0.436]	0.066 [0.033, 0.103]	0.598 [0.554, 0.640]	0.090 [0.059, 0.124]	0.454 [0.410, 0.498]	0.057 [0.029, 0.089]
Pooled	0.275 [0.254, 0.298]	0.052 [0.031, 0.074]	0.585 [0.561, 0.609]	0.101 [0.082, 0.122]	0.624 [0.602, 0.648]	0.125 [0.105, 0.146]

pooled at realised risk 0.101 (CI 0.082–0.122); M1 answers 0.033 at risk 0.220 (CI 0.109–0.339). The gate alone answers 0.624 at risk 0.125 (CI 0.105–0.146). The realised risk of M3 at $\alpha = 0.05$ is 0.043 (Python), 0.037 (Rust) and 0.066 (math), with CIs that cover 0.05 in each case. We claim no pass/fail on “meeting the target”; the CIs are wide and the plan says so in advance.

Caveats.

1. CIs resample source-problem clusters of *fixed* out-of-fold predictions. They do not include variability of the model fit.
2. ECE percentile-bootstrap CIs sometimes exclude their own point estimate. For example, for M2 on 4B Python the point estimate is 0.056 and the CI 0.057–0.111 (`disc.ece.M2.python`); the same happens for M2 on Rust (0.045, CI 0.047–0.101). This is the known upward bias of ECE under resampling, so the CIs of ECE are not read as symmetric intervals around the estimate.
3. It is one run on the burned E set, public benchmarks, one greedy sample, and not the registered GL.

6.8 A12: a Qwen3.5-9B tier and wider-gap cascades (revision 3.5.0; post hoc, exploratory)

Setup. Addendum A12 was written before the 9B run completed and with no 9B result seen. It restores the registered ladder’s 9B tier (the 27B was not attempted; budget). Qwen3.5-9B, bf16, vLLM-TPU, one `v5litepod-4` slice with tensor parallelism 4, greedy, same raw protocol, prompts, stop sequences and log-probabilities as the 2B/4B, answers for all 1,536 E tasks and, for math, a program per task (A10). A 6-task-per-domain smoke test (18 tasks) was not analysed. Chip-seconds are the slice wall time split by token share times the 4 chips in use (D45), so they are in the same unit as the single-chip 2B/4B but the 9B effective concurrency was not measured. The ledger records 587s (smoke) and 954s (full) for the slice, \$0.7827 and \$1.272, about \$2.05 in total (list price, Section 10). Cascades are replayed offline from the cached generations with the same ladder rule as the Study’s `gwenlaya` arm (first tier whose executed gate verifies, else the last tier, unanswered); the replay reproduces the Study’s own `gwenlaya` rows exactly (1,536 tasks, 0

mismatches in correctness, answered, escalated and cost; `meta.validation`), so no cascade was re-scored.

The 9B alone (Table 12). Pooled pass rate 0.653 (CI 0.630–0.676); by domain 0.726 (Python), 0.588 (Rust), 0.642 (math), against 0.675, 0.529 and 0.612 for the 4B. Math generations cut at the token limit: 0.306 (pooled share over all tasks 0.100; none in Python or Rust). Raw confidence remains badly calibrated (ECE 0.207, 0.344, 0.304 by domain). The executed gate on the 9B gives coverage 0.666 pooled, precision 0.885 and confident-wrong rate 0.077, and at matched coverage the gate’s rate is below the log-prob baseline by 0.053 (Python), 0.126 (Rust) and 0.082 (math) (all CIs below zero; `A4.delta_cwr_gate_minus_b3.qwen3.5-9b-bf16.*`). An oracle that picks the cheapest correct tier among 2B, 4B and 9B reaches 0.726 pooled (CI 0.704–0.748), against 0.653 for the 9B: a ceiling for any router over these three tiers on these tasks, not a comparator.

Reading rule, fixed before the results. A cascade *pays off* only if its cost ratio against always-9B is below 1 in both the with-warm-up and the without-warm-up readings and the CI of its accuracy difference does not exclude zero on the harmful side. Anything else is reported as not paying off. This is a descriptive rule, not a test, and there is no multiplicity correction across the twelve domain-by-cascade cells.

Results (Table 13).

- **4B→9B pays off for Python and Rust, not for math and not pooled.** Python: cost ratio 0.601 (CI 0.553–0.656) with warm-up and 0.749 (CI 0.700–0.800) without; accuracy difference +0.013 (CI −0.013 to 0.040); escalated share 0.187. Rust: 0.879 (CI 0.807–0.952) and 0.947 (CI 0.895–0.998); +0.081 (CI 0.053–0.110); escalated 0.404. Math: 1.806 and 2.306, with the program charged for each tier consulted, so it does not pay off even though accuracy is higher by +0.018 (CI 0.008–0.030). Pooled: 1.381 (CI 1.306–1.458) and 1.724 (CI 1.636–1.812), with accuracy higher by +0.037 (CI 0.023–0.051; 86 against 29 discordant tasks, Holm over the three cascades $p = 2.9 \times 10^{-7}$). The pooled cascade is more accurate than always-9B (0.690 against 0.653) but not cheaper.
- **2B→9B pays off for Rust only.** Rust: 0.909 and 0.892, accuracy difference +0.018 (CI −0.006 to 0.041). Python is cheaper (0.668 and 0.681) but fails the accuracy condition: −0.042 (CI −0.068 to −0.015). Math: 1.711 and 2.017. Pooled: 1.344 and 1.526, accuracy −0.008 (CI −0.021 to 0.004; 40 against 53 discordant tasks, Holm across cascades $p = 0.213$).
- **2B→4B→9B.** Python fails on accuracy (−0.038, CI −0.068 to −0.008) although it is cheaper (0.563 and 0.648). Rust meets the rule by point estimates (0.916 and 0.952, accuracy +0.077) but the CIs of both ratios include 1 (0.827–1.005 and 0.882–1.019), so we do not read this as a robust saving. Math 2.110 and 2.639. Pooled 1.565 and 1.905, accuracy +0.018 (CI 0.002–0.033; 91 against 64 discordant tasks, Holm across cascades $p = 0.073$).

In short, a wider tier gap makes the gate cascade cheaper than always-largest for the code domains (4B→9B for Python and Rust), but math, which carries the program cost of every tier consulted, and the pooled view still cost more than always-9B.

Caveats. The 9B effective concurrency was not measured, so its per-task cost is less certain (we do not know in which direction the cascade-to-9B ratios would move). The 4-chip unit is a slice wall-time split by token share. First-chunk warm-up is treated by the with/without readings only. The programs’ CPU time and the code gates’ CPU time are not counted. Per-domain rule outcomes have no multiplicity correction. E is burned for H1/H3, the benchmarks are public, there is a single greedy sample, and nothing here involves Laya.

Table 12: A12, Qwen3.5-9B (bf16, v5litepod-4, TP=4) alone. Pass rate, share of generations cut at 1,024 tokens, executed-gate coverage, precision and CWR, and the oracle router over the three tiers (accuracy, and its chip-seconds per task, picking the cheapest correct tier; a ceiling, not a comparator). 95% cluster bootstrap CIs.

Domain	pass rate	truncated	gate coverage	gate precision	gate CWR	oracle acc.	oracle cost
Python	0.726 [0.688, 0.764]	0.000	0.870 [0.841, 0.898]	0.833 [0.798, 0.866]	0.146 [0.115, 0.176]	0.784 [0.749, 0.819]	0.242 [0.213, 0.274]
Rust	0.588 [0.544, 0.631]	0.000	0.649 [0.607, 0.690]	0.906 [0.873, 0.936]	0.061 [0.041, 0.083]	0.708 [0.669, 0.748]	0.329 [0.286, 0.375]
Math	0.642 [0.598, 0.684]	0.306	0.468 [0.424, 0.510]	0.957 [0.930, 0.982]	0.020 [0.008, 0.032]	0.682 [0.640, 0.722]	0.933 [0.807, 1.075]
Pooled	0.653 [0.630, 0.676]	0.100	0.666 [0.644, 0.688]	0.885 [0.865, 0.903]	0.077 [0.064, 0.090]	0.726 [0.704, 0.748]	0.496 [0.450, 0.545]

Table 13: A12, gate cascades versus always-9B, with the reading rule fixed in addendum A12. “Pays off” needs a cost ratio below 1 in both the with-warm-up and no-warm-up readings (point estimates) and a CI of the accuracy difference that does not lie wholly below zero. 9B effective concurrency was not measured, so its cost is less certain. Point [95% CI].

Cascade	Domain	Δ acc. (casc. - 9B)	cost ratio	cost ratio, no warm-up pays off?
2B→9B	Python	-0.042 [-0.068, -0.015]	0.668 [0.598, 0.740]	0.681 [0.624, 0.737]
	Rust	+0.018 [-0.006, +0.041]	0.909 [0.841, 0.975]	0.892 [0.843, 0.938]
	Math	+0.000 [-0.008, +0.008]	1.711 [1.620, 1.802]	2.017 [1.905, 2.129]
	Pooled	-0.008 [-0.021, +0.004]	1.344 [1.285, 1.403]	1.526 [1.452, 1.602]
4B→9B	Python	+0.013 [-0.013, +0.040]	0.601 [0.553, 0.656]	0.749 [0.700, 0.800]
	Rust	+0.081 [+0.053, +0.110]	0.879 [0.807, 0.952]	0.947 [0.895, 0.998]
	Math	+0.018 [+0.008, +0.030]	1.806 [1.680, 1.938]	2.306 [2.173, 2.439]
	Pooled	+0.037 [+0.023, +0.051]	1.381 [1.306, 1.458]	1.724 [1.636, 1.812]
2B→4B→9B	Python	-0.038 [-0.068, -0.008]	0.563 [0.505, 0.629]	0.648 [0.581, 0.715]
	Rust	+0.077 [+0.043, +0.110]	0.916 [0.827, 1.005]	0.952 [0.882, 1.019]
	Math	+0.016 [+0.004, +0.030]	2.110 [1.960, 2.268]	2.639 [2.463, 2.815]
	Pooled	+0.018 [+0.002, +0.033]	1.565 [1.470, 1.661]	1.905 [1.787, 2.024]

Table 14: Deviations D31–D41 of revision 3.4.0 (full text in docs/DEVIATIONS.md). “Post hoc” = decided after exploratory results were seen.

#	Registered	Done here	When
D31	L4 GPU, llama.cpp	one TPU v5e chip, vLLM-TPU 0.31.0, bf16;	before genera-
D32	(D6: llama.cpp)	weights at HF revision main (not pinned)	tion
	tiers 2B/4B/9B/27B	2B/4B only (“largest” = 4B), plus Qwen2.5-Coder-	before genera-
		1.5B as a continuity anchor	tion
D33	one engine configuration	configured max_num_seqs 96 (Coder, 2B) and 32	during genera-
		(4B; 96 failed with “Cannot fit both KV pools”);	tion
		effective KV capacity at most 54 (2B) and 11 (4B)	
		concurrent requests	
D34	H3 cost in L4 GPU-seconds	accelerator chip-seconds, chunk wall time split	plan
		by token share, including first-chunk warm-up;	
		throughput, not latency	
D35	B3 threshold set on split C	A4 baseline answers the k most confident tasks, k	plan
		= gate’s count in the same domain (no labels used;	
		pessimistic ties); matched on E, not frozen on C;	
		not B3	
D36	GL = gate + Laya; H3 vs B5	cascade = gate ladder without Laya, compared	plan
		with B1/B2; not GL, not H3	
D37	math gate = program-of-	not implemented in 3.4.0: our gate executed only	plan (stated)
	thought re-execution	gate_payload , empty for 500/500 math rows; math	
		gate coverage 0. Superseded by D43 in 3.5.0	
D38	–	A7 restricted to Python/Rust (CPU night math de-	post hoc
		fective)	(amend. 1)
D39	–	serial overlay for contention flips; passing-row re-	post hoc
		run; S1 fail-closed sensitivity; nondeterministic	(amend. 2, 4)
		item stays FAILED	
D40	logprobs captured (D21)	first TPU pass returned no log-probabilities; set	during genera-
		aside and fully regenerated with logprobs=1	tion
D41	sequential checks	2B/4B scored with 5 parallel check workers (Coder:	scoring
		one process); contention timeouts corrected by D39	

Table 15: Deviations D43–D47 of revision 3.5.0 (full text in docs/DEVIATIONS.md). All were decided after the E baselines had been seen.

#	Registered / before	Done here	When
D43	math gate = program-of-thought re-execution (D37: not implemented)	implemented and run on E math for the 2B and 4B after the zero math coverage was seen (A10); programs charged to every gated arm per tier consulted; math rows of the overlay replaced by the replay	post hoc
D44	tiers 2B/4B/9B/27B (D32: 2B/4B only)	Qwen3.5-9B run on all 1,536 E tasks plus math programs (A12); 27B not attempted; reading rule for “pays off” fixed before results	post hoc; A12 written before the 9B run completed
D45	cost in L4 GPU-seconds (D34: chip-seconds)	9B on a v5litepod-4 slice, TP=4; chip-seconds = slice wall time split by token share $\times$ 4; 9B effective concurrency not measured	post hoc
D46	calibrator fitted on split C (no Laya, no C)	A11 cross-fitted logistic abstention on E, run once; M3 uses the gate as a feature; run for 4B and 2B only	post hoc
D47	–	ledger now 11 lines, \$4.9136 (list price); three lines added since 3.4.0	recorded

6.9 Deviations recorded for this section

7 Hypothesis status

Table 16 restates docs/RESULTS_VERDICT.md. The probability column is the **red-team prior** written before any data (docs/PROPOSAL_REVIEW.md, copied into docs/DEVIATIONS.md). It is *not* an estimate from data and was not updated by any result in this paper. No exploratory analysis of Section 6 is a verdict on a registered hypothesis. Where an exploratory analysis is related, the table says so and why it is not the test.

H1 and H3 cannot be run as registered on these E generations. The registration’s freeze rule (Section 6) requires the calibrator, all thresholds and the E manifest hash to be fixed in a signed addendum *before E is generated*, and it forbids E-based refitting. E has now been generated, and its outcomes analysed and published, before split C or the Laya head exist. A registered test therefore needs one of two things:

- fresh generations on a new evaluation set E' after the freeze (for example, problems released after the models’ training cutoff, which would also reduce contamination); or
- a recorded deviation stating that the E generations predate the freeze and that their outcomes were seen, in which case any H1/H3 run on them is a deviated test.

H13 (budget). The decision rule has two parts: the ledger total must be $\leq$ \$40 planned and $\leq$ \$50 hard, and no stage may exceed $1.25\times$ its cap twice. The ledger total read for this revision is \$4.9136 (Section 10), so H13 is not refuted on an interim basis. Three caveats apply:

1. The GPU stage launcher is designed to refuse a stage that would push the ledger over its cap, so H13 was expected to hold largely by construction (red-team prior 0.75, “close to tautological”).
2. The TPU runs recorded up to v3.3.1 were launched by separate scripts with a run-time limit but no ledger check, so the low total reflects low usage. Since commit 39a5a0a, the committed driver `deploy/tpu/drive.sh` is a dry run by default. Before it creates anything, it checks the ledger total plus the run’s worst case against a cap. The driver log of the second E generation run shows this check: ledger \$1.9809, worst case \$2.5, cap \$45 (`egen/drive_e_gen2.log` in the session scratchpad; not committed and not in the lake). I did not see the driver log of the first E run, so I do not claim that run was checked. I also

Table 16: Hypothesis status after revision 3.5.0 (no status changed by the new analyses). “Prior” = red-team prior that the hypothesis would be supported, written before any data; not updated by evidence.

H	Role	Status	$P(\text{true})$	Basis
H1	primary	not run	0.45 (prior)	needs GL (Laya) and B3 with threshold on C, frozen before a fresh E'; A4 and A10 are exploratory analogues (gate without Laya, k matched on E; 3.5.0 adds math), not H1
H3	primary	not run	0.50 (prior)	needs GL and B5, frozen before a fresh E'; A5, A10 and A12 compare gate cascades without Laya with B2 (and with always-9B) in chip-seconds, not H3
H4	secondary	not run	0.15 (prior)	no Laya head; A2 is raw confidence only; A11 is a cross-fitted logistic score, not the Laya head
H5	secondary	not run	0.25 (prior)	no Laya head (A11 does not test it)
H6	secondary	not run	0.40 (prior)	no Laya calibrator (A11 is an exploratory cross-fitted substitute on burned E)
H8	secondary	not run	0.30 (prior)	needs GPU (D11); A7 is not a quantisation test
H9	secondary	deferred	0.25 (prior)	D8
H13	systems	not refuted (interim)	0.75 (prior)	ledger \$4.91 (list-price estimate); not evidence about the system, Sec. 7
H14	systems	not run	0.50 (prior)	needs E1 (D12)
H15	systems	not run	0.70 (prior)	offline test is not the registered SIGKILL check
H2	exploratory	not run	0.08 (prior)	needs GL and B5
H7	exploratory	not run	0.10 (prior)	registered arms absent; descriptive analogue A3/A10 (4B gate, TPU) has precision 0.830 Python, 0.887 Rust, 0.943 math (math CI includes 0.95), point estimates below the 0.95 flag
H10	exploratory	dropped	0.05 (prior)	D8
H11	exploratory	dropped	0.12 (prior)	D7, D8
H12	exploratory	dropped	0.12 (prior)	D8

did not inspect driver logs for the program-of-thought and 9B runs of 3.5.0, so I make no claim that they were ledger-checked.

3. In either case it is not evidence about the GwenLaya system. H13 is a whole-study claim, the planned GCS ledger `runs/LEDGER.jsonl` still does not exist (read-only listing), and the verdict is interim.

H15 (preemption). `tests/test_run_study.py::test_resume_after_kill_mid_run` passes, but it simulates a death by appending a torn line. It sends no SIGKILL, so the probability stays at the prior.

8 TPU feasibility findings

- **Qwen2.5 works.** In smoke run #1 (on demand, us-central1-a; logs under `gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/tpu_smoke/gwenlaya-tpu-smoke-215529/`), Qwen/Qwen2.5-Coder-1.5B-Instruct loaded and generated with vLLM 0.31.0’s TPU backend: load 208.9 s, then 442 completion tokens in 0.61 s.

The gate on that VM returned UNVERIFIED (no bubblewrap there), which is the designed fail-closed behaviour.

- **Qwen3.5 0.8B, 2B and 4B generate on one v5e chip with lazy compilation.** In run #1, Qwen/Qwen3.5-4B did not finish loading within 1,500 s. Qwen3.5 is not JAX-native in tpu-inference and runs through the PyTorch fallback. It runs when up-front precompilation is skipped (SKIP_JAX_PRECOMPILE=1). With that setting:
 - Qwen3.5-0.8B generated 16 night tasks in run #2 (results/gwenlaya_v4/tpu_run2/res_qwen35_0p8b.jsonl.summary.json). The run #2 scores are not used, because its TPU-host gate self-test reported `python_ok False`.
 - Qwen3.5-2B and 4B generated all 80 night tasks in run #3.
 - The same two models generated all 1,536 E tasks in this revision.
- **Concurrency limit for the 4B.** On the 16 GiB chip the 4B (hybrid full/linear attention; vLLM sizes a “mamba” recurrent-state pool) failed to start with `max_num_seqs 96` (“Cannot fit both KV pools”). It started at 32, where the engine re-split the KV cache for 11 concurrent requests. The 2B ran at 96 configured and 54 effective (Section 6).
- **Log-probabilities need logprobs=1.** The first E pass returned none (Section 6). The committed generator requests `logprobs=1` and fails fast if none come back.
- **No TPU LoRA path was found for Qwen3.5 dense or Qwen3.8** in the stacks we inspected. Serving Qwen2.5 with `enable_lora` failed at engine initialisation, and this is the only vLLM-TPU LoRA attempt. Tunix 0.1.7 lists Qwen2/Qwen3; MaxText lists Qwen2.5/Qwen3 and Qwen3.5 mixture-of-experts scripts. The training steps (Laya head, SFT) therefore stay on the GPU/CPU plan.

9 Descriptive TPU run #3 (history; not a registered arm)

What ran. TPU run #3 used one on-demand TPU v5e chip (us-west4-a), vLLM-TPU 0.31.0, bf16 weights, greedy decoding, `max_tokens 1024` and non-thinking mode, with the D28 math prompt and TPU-specific code system prompts (via `llm.chat`, with no code prefill). It generated the 80 night tasks for three models (results/gwenlaya_v4/tpu_run3/). Scoring on the TPU host failed for every Python item, for a reason that was not recorded (the gate self-test on the same host passed). All rows were therefore re-scored on our machine.

Table 17: Descriptive TPU run #3 (one v5e chip, bf16, greedy, one sample, chat prompts; not a registered arm, not a test). Items passing the checks out of 28 Python, 26 Rust and 26 math items; “trunc.” = math items cut at 1,024 tokens (all UNVERIFIED). Load, generation time, tokens and tok/s are the run’s own self-reported **SUMMARY** fields.

Model	Python	Rust	Math	trunc.	load s	gen. s	tokens	tok/s
Qwen3.5-2B	11/28	6/26	16/26	9	106.2	99.7	25,568	256.3
Qwen3.5-4B	20/28	13/26	16/26	9	124.3	117.4	25,852	220.2
Qwen2.5-Coder-1.5B-Instruct	18/28	15/26	8/26	4	186.3	8.7	18,219	2,094.2

Reading (corrected wording). Qwen3.5-2B in bf16 under vLLM on TPU, with a different code system prompt and no code prefill, gives the same Python and Rust pass counts as the q4 CPU arm. Item agreement is 22 of 28 for Python and 24 of 26 for Rust. Hardware, quantisation, engine and code prompt all changed, so this is a loose consistency check, not an isolation of any one factor. The same-protocol comparison A7 (Section 6.5) supersedes it.

10 Actual GCP spend

The figures below are read from `/mnt/data/home/xavkal/gwaya-data/spend_ledger.jsonl`. The ledger was last written at 2026-10-09 09:37:29 and read for this revision on 2026-10-09 (sha256 in Section 11). It has 11 lines, and the sum of `usd_estimate`, recomputed, is 4.9136. Revision 3.4.0 showed 8 lines and 2.5289; the three lines added since are the last three rows below, totalling 2.3847.

Time	Item	Seconds	USD
2026-10-07 21:54	TPU smoke, attempt 1 (spot, us-west4-a; pre-empted)	240	0.0329
2026-10-07 22:38	TPU smoke (on demand, us-central1-a)	2,466	0.8220
2026-10-08 00:35	L1 upload to data lake (6 objects)	13	0.0
2026-10-08 07:09	TPU run #2 (on demand, us-central1-a)	1,073	0.3577
2026-10-08 07:36	TPU run #3 (on demand, us-west4-a)	982	0.3273
2026-10-08 21:36	E generation, first pass (on demand, us-west4-a; no log-probabilities, set aside)	963	0.3210
2026-10-08 21:44	orphan VM : created 21:38:23 by a driver the maintainer killed by mistake, deleted 21:44:35 (us-central1-a)	360	0.1200
2026-10-08 22:20	E generation, second pass, used here (on demand, us-west4-a)	1,644	0.5480
2026-10-09 08:47	program-of-thought generation, 2B and 4B (on demand, us-west4-a, v5litepod-1)	990	0.3300
2026-10-09 09:07	9B smoke test (on demand, us-west4-a, v5litepod-4), 18 tasks, not analysed	587	0.7827
2026-10-09 09:37	9B full run (on demand, us-west4-a, v5litepod-4)	954	1.2720
Total (sum of <code>usd_estimate</code>)			4.9136

All non-zero spend is TPU use, priced from Cloud Billing Catalog list prices: spot v5e \$0.494237 and on-demand v5e \$1.20 per chip-hour, as recorded in each line's `price_source`. All lines are v5litepod-1 (one chip) except the two 9B lines, which are v5litepod-4 (four chips, so seconds $\times 4 \times 1.20/3600$). The two 9B lines together cost \$2.0547, about \$2.05.

- The three E-generation lines (first pass, orphan VM, second pass) total \$0.989. About \$0.44 of that (first pass plus orphan) produced nothing used here.
- The orphan line is an upper bound, since billing may start only when the VM is ready.
- The billed amount is TBD until the billing export is read.
- All scoring, re-checking and analysis ran locally and cost nothing on GCP.

11 Reproducibility

Code.

- **Earlier releases.** v3.2.0 = 825d1ba. Tag v3.3.0 points to commit 200c7f4, the merge of the night branch that contains ebd032f. v3.3.1 = 4914b5c on main, with the Rust fix PR #4 (8cb37ad) and the math/Python fixes 2c1f6b1.
- **This revision** is on branch `bench/tpu-e-eval`:
 - 39a5a0a: TPU batch generation, importer, capped driver, the earlier TPU and validation scripts, and the claim audit.
 - 66e2313: memoised and optionally parallel checks.
 - 8ee9da3: analysis plan.

- 5e10ad2: analysis script.
- 40dcae5, 5ef8e43: overlay, A7 restriction, S1 and the plan amendments.
- f80ac16: results.
- **Revision 3.5.0** is on branch `feat/math-gate` (HEAD 227eaf8 when this text was written; the paper edits of 3.5.0 are not yet committed):
 - 803f350: program-of-thought math gate, charged for its generation (A10).
 - 4af1fe9, fa8826b: plan addendum A11 (written before fitting) and its analysis script.
 - cb5c6b5: chips-aware chip-seconds for multi-chip slices; plan addendum A12 written before results.
 - 227eaf8: A10, A11 and A12 results.
 - `scripts/make_tables_350.py` writes `papers/tables_a10_a12_part1.tex` to `part3.tex` from the three result JSON files (not committed yet).
- **Night CPU arm.** llama.cpp release b11476 (CPU, asset sha256 2cda5ff9...4b5e). Qwen3.5-2B-Q4_K_M.gguf from unsloth/Qwen3.5-2B-GGUF revision f6d5376b..., sha256 aaf42c8b...9223; 4B revision e87f1764..., sha256 00fe7986...11a4 (all in L0_env.json).

Frozen inputs and results (sha256, all recomputed for this revision).

File	sha256
<code>experiments/night/manifest_eval.json</code>	bd1684dadcf38328fb6fab7619e5ea2f4fbc5ccc0e31022e059360b2d8c615e0
<code>tasks_E_night.jsonl</code> (80 items)	e842edd13951b4082c4495f7bcc845b032989f4a457e0c5d6e9c44f2b72cd679
<code>tasks_E_primary.jsonl</code> (1,541 items)	971f44c61bad5044f8487383f8a3ea27bc2c292c565a6ff8ef1254fce1cc7097
<code>docs/FREEZE_ADDENDUM_NIGHT_2026-10-07.md</code>	ac2970983ef45eb027de65194d42cadd41483dfd35cd2a866efc64247b02376c
<code>experiments / night / decontam_report.json</code>	d128e06c5949c86c29537f71a1d547ef953be6aa22f7bc814a280ec4d48f1907
<code>night/cache/L2score/rows.jsonl</code>	1e35fac792d29d8f82ce05f5cbad0d615bb4db2a6a707d157edff75da0fa69c6
<code>night/cache/L2score/gens.jsonl</code> (= L2fix)	03f6e2b6af9cebd65d8488b9a35a1f7d3bc70b566397379edc3d95f5291963db
<code>papers/numbers_v4.json</code> at tag v3.3.0	fccb1695538e76eba93669d1a232d24c7373e8aa2ad37d900f0e19c153275995
<code>night / cache / L2fix / rows . jsonl</code> (v3.3.1 re-score)	a5e72614681a81acf379050270677eead4844e4833494c32f7695c485cae70a8
<code>papers/numbers_v4.json</code> at tag v3.3.1 and HEAD	5f3c2a6e1e069663af306f67f4e2ba0e32155044908cba35fd5826f092d8a1c0
<code>results/gwenlaya_v4/rust_harness_validation.json</code>	93291a5a56c050d15fbb98a501e16d575251602e30507e563ff610f24583e8d8
<code>results / gwenlaya_v4 / tpu _ run3 / summary.json</code>	e38310f6b4e7869c0f84a3cfad874e0820cd77a29f81e173f6e4c0cc80f94648
<code>tasks_E_night.d25.jsonl</code> (80, D25)	504909fc74dc2af8fb3f2491a475df07de8e95e6fdf0f4f69bc22352e95e0952
<code>tasks_E_primary.d25.jsonl</code> (1,536, D25)	2143110c78a3a9f57599dce77021cf4571fc57cf2584017f16ebdd08d5acba66
<code>docs / ANALYSIS_PLAN_E_TPU . md</code> at 8ee9da3 (original)	1f2fa2c11cf5fa52f12c5864f63a61b9b4ddd1ad8086a1946105f9b046020d46
<code>results / gwenlaya_v4 / e _ tpu / numbers_e_tpu . json</code> (regenerated after the re-audit; working tree)	9197f367f53d28efce0aa6c7de01dd83e67c786a20c4232fc35b07d6171547bc

numbers_e_tpu.json as committed in f80ac16 (superseded)	c76bc506c916c8b78286f4fdfeabc271e3eb58a1fdb0a3d211d4a965e32226b
papers/tables_e_tpu.tex (regenerated)	1aa86421460c261abbfc5e12171975de4497e032d4d6a83731b0516c6aaa58eb
e_tpu/data/rows_tiers.jsonl (= etpu/tiers/rows.jsonl)	b1bf6df59f05a03f771ac4cf667d74da90f6889f3129dfb4d750df29a1406277
e_tpu/data/rows_coder.jsonl	6898d5ae60c20fd1a1defe5be2bb0b5337b4832f9710074bc707b8fc1bae82a9
e_tpu/data/gens_summary.jsonl	46d02306bc84ad9749f384aaaf7b6cb09f0f96ffe503abd9ce8bcb0d9e41c98c
e_tpu/overlay/rows_overlay_tiers.jsonl	54e9ff405358fd7d090e0eba0723f156c6a168304ab326f2e2729ac04b0a94b4
etpu/tiers/gens.jsonl (data disk)	b6362df66cc8b7ffd5177a7eeb8866cf8f0a7d129d70074e4daac46783875fce
raw_qwen3.5-2b-bf16.jsonl (lake, second pass)	e077aa1da6a36e7a0b0de8748e720de5315c0c836dff0c90e307c4e51a201960
raw_qwen3.5-4b-bf16.jsonl (lake, second pass)	d2e1847fff7b19be35e2f20a6fefe246e3bd66dec55dc08b9bdf51e6671970ab
raw_qwen2.5-coder-1.5b-bf16.jsonl (lake, second pass)	df6ee0d466195023526e126abb862d73cb45b2dd969ba8d3215fef24d4ef3c42

Revision 3.5.0 (all recomputed with sha256sum on 2026-10-09)

results / gwenlaya_v4 / e_tpu_mathgate/numbers_e_tpu_mathgate.json	ce662f842e64058687f659121fec2979264a587aba68448710d73eebe69d9a1b
results / gwenlaya_v4 / e_tpu_mathgate/abstention_a11.json	9ee7d4dd0da18fb1d10aeb8c7fc4772c186de2d10d05a6e4450b5ebf321fc44c
results / gwenlaya_v4 / e_tpu_mathgate/cascades_a12.json	5d331a630027ab874e237f9e3cbfe387a79ad862a9cfb4ed90a31c47d24e59c8
results / gwenlaya_v4 / e_tpu_mathgate/tables_e_tpu_mathgate.tex (generated, not input)	163944211ac263795b86ba4b378d01128f6bd6f17ff7e39a360d616750c6d8d0
results / gwenlaya_v4 / e_tpu_mathgate/rows_mathgate_2b4b.jsonl (= etpu/mathgate/rows.jsonl)	2a27e0926956d4b4152cd39d4ad22f63641fb3dfb760896335a6018881f0d9de
results / gwenlaya_v4 / e_tpu_mathgate / rows_9b.jsonl (= etpu/nine/rows.jsonl)	5303da0da3ecda8259a82b3529980c7300803341968ab39f4ece110df54aa7b2
results / gwenlaya_v4 / e_tpu_mathgate / figures / e_risk_coverage.png (= papers/figures_v4/e_risk_coverage_a10.png)	75fce2e39433195424ff844f49f791e6870827b4356cecef52cd9b58ff6c7ff9
results / gwenlaya_v4 / e_tpu_mathgate / figures / e_cascade_pareto.png (= papers/figures_v4/e_cascade_pareto_a10.png)	7a3f142a9989afa167e3628a7c2988267b2abd57671317f6ee4a09a59554d19b
gwaya/domains/math_gate.py	6fe1d5a3c2035a355c24b572a8937d2321dee57c8e4a3fe5a1e3db628c58c032
scripts/analyze_abstention.py	900dd40b6c0ce277f1c1f11fa5a457f88be4319730f12cbde87137fc1848788d
scripts/analyze_cascades.py	4e137d9f4fcd65b5477c2668b8d5f9ceb4742d4d179179107d115397bb26f731
scripts/make_tables_350.py	bba450ffeaaffea6f90dc3da63d56f3dabab0d9e75bba95b0d5b65ee292433244
papers/tables_a10_a12_part1.tex	4522326e346ae5271b1fb6c1dc42318d76100a13adbf9a1eac781c1d691d27df
papers/tables_a10_a12_part2.tex	9949d94bc2b345465e3f6a2581a71a63fab0bc4c165722443c0211bb3e68cfb6
papers/tables_a10_a12_part3.tex	3e544f5bd3a0e59ad7e63f511253460ddd7ac8bba6727f4575d3d2b4bc5efa02
docs/ANALYSIS_PLAN_E_TPU.md with addenda A10–A12 (at cb5c6b5; the A1–A9 text is unchanged)	92a3f85ce11518b8ba2f32c881d51bfe319ffed738a4703d2f89c4a27c135ce9
docs/DEVIATIONS.md with D43–D47 (working tree)	dac58d8dddbb78fa6182699befd991990d838020009172a3440bbfd44fae88aa

spend_ledger.jsonl (11 lines, read for 3.5.0; a living file)	ff882615f769fd366afc6c10d3d2f7ed4e52ffc60025ed9b19aa24143943c2d4
etpu/nine/gens.jsonl (data disk)	c01e317819425ff70d3101ec935d8fd04f9bebeecdb6e8ae8105b71e8866e3c6
etpu/mathgate/gens.jsonl (data disk)	3eef7582c6a4c7da1047d0e033ce59ade2e02c209606c50719dffbd ebf6d4661
etpu/nine_raw/raw_qwen3.5-9b-bf16.jsonl	2df7659949a3247be99d45471eebe1f7d4610c7d6228618fd4bfdbc a9f3edeab
etpu/nine_raw/raw_pot_qwen3.5-9b-bf16.jsonl	39c3f51533dce6802826124212e3d3c9a2e2bacf69cab68edf20a5c 989f8e64f
etpu/pot_raw/raw_pot_qwen3.5-2b-bf16.jsonl	adf0156e37f5b70c26dde6723992aec12f722a97d5fb7d0fdd1e51b a385a98ce
etpu/pot_raw/raw_pot_qwen3.5-4b-bf16.jsonl	3abe365ac3a83b72bdf8ae580b84ee27e6c27973bad8f0f20f2581c 93375dc26

The HF dataset revisions are in the freeze addendum (MATH-500 6e4ed1a2, humanevalplus d32357cf, mbppplus b2d74c91, MultiPL-E 28441b60). The E-set model weights were loaded at revision main and are not pinned.

Data lake (gs://socrateai-datalake-gen-lang-client-0625573011/gwenlaya_v4/; read-only listings).

- night/2026-10-07/: the night inputs.
- tpu_smoke/: smoke runs #1-#3.
- tpu_runs/gwenlaya-tpu-e-gen-210955/: the first, discarded E pass.
- tpu_runs/gwenlaya-tpu-e-gen2-214445/: the second E pass (raw generations, gen_batch logs, status.jsonl).
- e_tpu_final/: scored rows and generations of the tiers and the Coder, the overlay and the audit reports.

The night CPU rows are on the local data disk and in the repository (results/gwenlaya_v4/night_L2fix/). Mirroring them to gwenlaya_v4/results/, as the registration requires, is still open.

Commands (full E set). The environment is PYTHONPATH=repo, GWAYA_DATA_ROOT=data root, with GWAYA_ALLOW_UNISOLATED unset. The exact flags of each step are in the commit messages and script headers.

```

deploy/tpu/drive.sh --name e-gen2 --script deploy/tpu/run_e_gen.sh \
  --upload "repo.tgz tasks_prompts.jsonl" --download "raw*.jsonl status.jsonl gen*.log" \
  --max-minutes 120 # dry run unless --yes-spend
# on the VM: run_e_gen.sh -> scripts/tpu/gen_batch.py
# (SKIP_JAX_PRECOMPILE=1, max-seqs ladder 96 32 16 8)
python scripts/import_remote_gens.py --tasks tasks_E_primary.d25.jsonl --raw raw_<model>.jsonl \
  --served <model> --quant bf16 --out $D/etpu/tiers --accelerator "TPU v5e x1"
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 --backend openai \
  --backend-url http://127.0.0.1:1/v1 --tasks tasks_E_primary.d25.jsonl \
  --models qwen3.5-2b-bf16 qwen3.5-4b-bf16 --quants bf16 \
  --arms base always_smallest always_largest gate_only raw_confidence gwenlaya \
  --mode score --no-vram --check-workers 5 --out $D/etpu/tiers
# Coder: scripts/tpu/score_local.sh (import + score, arm base)
python scripts/tpu/recheck_failures.py --run $D/etpu/tiers --tasks ... --model <model> \
  --out recheck_<model>.json
python scripts/tpu/recheck_failures.py ... --verified --out recheck_pass_<model>.json
python scripts/tpu/serial_overlay.py --audits recheck*.json --raw-dir $D/etpu/raw --tasks ... \
  --out $D/etpu/overlay2 --tiers qwen3.5-2b-bf16 qwen3.5-4b-bf16 --coder qwen2.5-coder-1.5b-bf16 \
  --tiers-gens $D/etpu/tiers/gens.jsonl --coder-gens $D/etpu/qwen2.5-coder-1.5b-bf16/gens.jsonl
python scripts/analyze_e_tpu.py --tiers $D/etpu/tiers --coder $D/etpu/qwen2.5-coder-1.5b-bf16 \
  --tasks tasks_E_primary.d25.jsonl --night-cpu results/gwenlaya_v4/night_L2fix/rows.jsonl \
  --night-tasks tasks_E_night.d25.jsonl --unstable-audits ... --overlay-tiers ... \
  --n-boot 10000 --seed 0 --out-numbers results/gwenlaya_v4/e_tpu/numbers_e_tpu.json \
  --out-tables papers/tables_e_tpu.tex --out-figs papers/figures_v4

```

I did not re-run `analyze_e_tpu.py` for this revision. I verified that its committed output matches the hash above, and I recomputed the per-model pass counts and truncation counts from the committed rows.

Commands (revision 3.5.0). The exact command lines of these steps were not saved; below are the shapes, taken from the helper scripts `run_mathgate.sh` and `run_nine.sh` (data disk, etpu/) and from each script’s argument parser. *D* is the data root.

```
# A10: program-of-thought generation on TPU (2B, 4B), then import and score (arms incl. gate_only, gwenlaya)
python scripts/import_remote_gens.py --tasks $D/etpu/mathgate/tasks_math.jsonl --raw <answers>.jsonl \
  --served <model> --quant bf16 --out $D/etpu/mathgate --accelerator "TPU v5e x1" --seconds-from <tiers>/gens.jsonl
python scripts/import_remote_gens.py ... --raw $D/etpu/pot_raw/raw_pot_<model>.jsonl --kind pot
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 --backend openai \
  --backend-url http://127.0.0.1:1/v1 --tasks $D/etpu/mathgate/tasks_math.jsonl \
  --models qwen3.5-2b-bf16 qwen3.5-4b-bf16 --quants bf16 \
  --arms base always_smallest always_largest gate_only raw_confidence gwenlaya \
  --mode score --no-vram --check-workers 5 --out $D/etpu/mathgate
# A12: 9B on v5litepod-4 (TP=4), import with --chips 4, score arms base and gate_only
python scripts/import_remote_gens.py --tasks tasks_E_primary.d25.jsonl --raw $D/etpu/nine_raw/raw_qwen3.5-9b-bf16.jsonl \
  --served qwen3.5-9b-bf16 --quant bf16 --out $D/etpu/nine --accelerator "TPU v5litepod-4 (4 chips)" --chips 4
python scripts/import_remote_gens.py ... --raw $D/etpu/nine_raw/raw_pot_qwen3.5-9b-bf16.jsonl --chips 4 --kind pot
python scripts/run_study.py ... --models qwen3.5-9b-bf16 --arms base gate_only --mode score --out $D/etpu/nine
# analyses (flags from each argparse; paths as in the scripts’ headers)
python scripts/analyze_e_tpu.py --tiers $D/etpu/tiers --coder ... --tasks tasks_E_primary.d25.jsonl \
  --overlay-tiers $D/etpu/overlay2/tiers/rows.jsonl $D/etpu/mathgate/rows.jsonl --n-boot 10000 --seed 0 \
  --out-numbers results/gwenlaya_v4/e_tpu_mathgate/numbers_e_tpu_mathgate.json
python scripts/analyze_abstention.py --tiers $D/etpu/tiers --tasks ... --overlay <overlay2 rows> <mathgate rows> \
  --models qwen3.5-4b-bf16 qwen3.5-2b-bf16 --out results/gwenlaya_v4/e_tpu_mathgate/abstention_a11.json
python scripts/analyze_cascades.py --tasks ... --rows <tier rows incl. nine/rows.jsonl> --gens <gens incl. nine/gens.jsonl> \
  --out results/gwenlaya_v4/e_tpu_mathgate/cascades_a12.json
python scripts/make_tables_350.py # paper tables of 3.5.0
```

I did not re-run the TPU generations or the three analysis scripts for this revision. The numbers in Sections 6.6–6.8 were read from the three JSON files above (hashes in the table).

Commands (night CPU arm). With the environment of `night/run_L2.sh`:

```
python scripts/run_study.py --plan experiments/night/plan_night.json --stage night_L2 \
  --backend openai --backend-url http://127.0.0.1:8091/v1 \
  --tasks $NIGHT/results/tasks_E_night.jsonl --models qwen3.5-2b-q4_K_M \
  --quants q4_K_M --arms base --mode generate --no-vram --cpu-pid <llama-server pid> \
  --out $NIGHT/cache/L2
cp $NIGHT/cache/L2/gens.jsonl $NIGHT/cache/L2fix/ # v3.3.1: re-score only
python scripts/run_study.py ... --mode score --out $NIGHT/cache/L2fix
python scripts/analyze_study.py --results $NIGHT/cache/L2fix/rows.jsonl $NIGHT/cache/L2fix/gens.jsonl \
  --tasks $NIGHT/results/tasks_E_night.jsonl --ledger $GWAYA_DATA_ROOT/spend_ledger.jsonl \
  --n-boot 10000 --seed 0
python scripts/data/regen_gate_payloads_d25.py --results $NIGHT/results # D25 task files
```

12 Limitations

- **No registered test.** No pre-registered test ran. Nothing here supports or refutes H1–H12, H14 or H15.
- **Unregistered E-set baselines.** The full-E baselines are exploratory: single greedy sample, one chip, one engine version, bf16, two tiers, weights not pinned to a commit. Four of the plan’s analysis choices were amended after the first results (Section 6.1). The Coder rows existed 10s before the plan commit.
- **Matched coverage in A4.** The baseline’s *k* is matched to the gate’s coverage on E rather than taken from a threshold frozen on a separate split. The gate’s advantage also includes executing a visible test.
- **E is spent for registered H1/H3.** E was generated and analysed before the registration’s freeze (calibrator and thresholds), so H1/H3 cannot be run as registered on these generations. A registered test needs fresh generations on a new set *E'*, or a recorded deviation.

- **The cascade is not GL.** It has no Laya, is compared with B2 rather than B5, and is not costed in GPU-seconds. Its chip-second cost depends on engine concurrency (effective at most 11 for the 4B vs at most 54 for the 2B) and on whether the compilation chunk is counted (pooled ratio 2.021 with it, 2.023 without, with the math gate of 3.5.0 and its program charge; 1.357 and 1.243 in 3.4.0), and it excludes the CPU time of the gate.
- **Math gate (revision 3.5.0).** The math gate (program-of-thought re-execution) is now implemented (A10), replacing the 3.4.0 “not implemented” statement. It is post hoc, the program is written by the same tier as the answer (so verification is agreement of one model with itself), its 4B precision 0.943 is below the 0.95 flag point (CI includes it), and its cost is charged only to gated arms. Math accuracy is still a lower bound, because up to 40 % of math answers (2B) were truncated at 1,024 tokens.
- **9B tier and chip-seconds (A12).** The 9B ran on a four-chip slice and its effective engine concurrency was not measured, so its cost, and every ratio against always-9B, is less certain. The “pays off” verdicts use point estimates for cost and have no multiplicity correction. The 27B was not run. A11 was not run for the 9B.
- **A11 (calibrated abstention).** M3 uses the gate verdict as a feature, so its gain is calibration and not independent evidence. CIs do not include model-fit variability, and ECE percentile-bootstrap CIs can exclude their own point estimates. It is not the Laya head; no Laya training has been done.
- **Post hoc.** A10, A11 and A12 were all decided after the E baselines had been seen (D43–D47), on an evaluation set that is burned for the registered H1/H3.
- **Stability.** One re-run of each passing row cannot rule out unstable candidates. Contention and five genuine timeouts per model remain. Scoring ran from working trees with uncommitted changes.
- **Contamination.** All absolute scores are on public sets that Qwen3.5 has probably seen. Only paired within-model deltas are claimed by the registration. The contamination-stratified analysis asked for in review finding F12 was not done (TBD).
- **Night data.** The night math generations are invalid (D28). The Rust harness was validated after the night generations existed, on hand-written solutions (25 of 26 tasks covered). The HumanEval+ night gate payloads contained the hidden tests (D25), but no gate ran there.
- **Spend.** The USD spend (\$4.9136, 11 ledger lines) is a list-price estimate, not the billed amount. H13 is weak evidence (Section 7).
- **Not run.** Lean 4, the training arms, quantization and the endpoint did not run.

13 Next steps

1. Train the Laya head and build split C.
2. Freeze the calibrator and all thresholds, then generate a fresh evaluation set E' (for example, problems released after the models’ cutoff) and run GL, B3 and B5 on it. Alternatively, record a deviation if the existing E generations are reused; any H1/H3 run on them would then be a deviated test.
3. Validate the program-of-thought math gate (A10) on fresh data in E' , with a precision target set in advance (the 3.5.0 gate is implemented but evaluated only post hoc on the burned E).
4. Pin model revisions.

5. Report cost at equal engine concurrency, including a measured 9B concurrency, and read the 27B tier if the budget allows.
6. Replace the cross-fitted A11 score by the registered Laya calibrator on split C.
7. Read the billed TPU amount.
8. Regenerate the night math on CPU.

A null or negative registered result will be reported in the same form as this one.

References

- [1] Anastasios N. Angelopoulos and Stephen Bates. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511*, 2021.
- [2] Xavier Callens. GWAYA v3.1: Fail-closed gate & real toolchain verification for LLM code generation. Zenodo, <https://doi.org/10.5281/zenodo.23121788>, 2026. Version 3.1.0; its +8.0 point claim is retracted.
- [3] Federico Cassano, John Gouwar, Daniel Nguyen, Sydney Nguyen, Luna Phipps-Costin, Donald Pinckney, Ming-Ho Yee, Yangtian Zi, Carolyn Jane Anderson, Molly Q Feldman, Arjun Guha, Michael Greenberg, and Abhinav Jangda. MultiPL-E: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.
- [4] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems*, 2017.
- [5] ggml-org. llama.cpp. <https://github.com/ggml-org/llama.cpp>, 2026. Release b11476, CPU build.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, 2017.
- [7] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS Datasets and Benchmarks Track*, 2021.
- [8] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [9] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- [10] Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024.
- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023.
- [12] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947.

A Claim audit of v3.3.1 (summary of docs/CLAIM_AUDIT.md)

The audit was run on 2026-10-08 on branch `bench/tpu-e-eval`, independently of the author. The auditor changed no paper, number, code or data file. Every number was recomputed from raw rows, generations, the ledger and logs (or the audit row says it was not), without using `numbers_v4.json` as a source.

Status	Count	What changed in 3.4.0
VERIFIED	61	nothing needed
VERIFIED_WITH_NOTE	29	notes on binning (A5), run #2 (D12, B8), assert counting (B3), registration timing (D11), L2 wall time (A30), self-reported timers (A24), the LoRA scope (B14), the Laya model (C14) and MultiPL-E metadata (C2) are now in the text (Section 2); bootstrap Monte-Carlo noise on two ratio upper bounds (A10, A14) needs no change
NOT_VERIFIABLE	2	B9 (cause of the run #3 TPU-host Python failure) now stated as unrecorded; C16 (pretraining exposure) is hedged and unchanged
WRONG	4	A32, A33 (hashes), A28 (stale spend), C12 (bib title): all fixed
OVERSTATED	3	B15 (H13), B12 (CPU-vs-TPU): reworded with the audit’s text; C17 concerned the README and was fixed there
Total	99	

By class: A, own quantitative results (45 rows), all 14 pass/fail counts correct, and re-running the checkers on the stored generations reproduced every stored status (0 mismatches over 80 CPU and 240 TPU items); B, code and system (20); C, external facts and citations (19); D, logic and statistics (15).

Where this revision differs from the audit. The audit gives the commit of tag `v3.3.0` as `ebd032f`. `git rev-parse v3.3.0^{commit}` gives `200c7f4`, the merge commit that contains `ebd032f`. The file hash the audit asked for (`fccb1695...`) is the same at both commits, so the fix is unaffected. I found no other disagreement.

B Machine-generated tables of the night arm

The tables on the next page are `papers/tables_v4.tex` as written by `scripts/analyze_study.py` from `papers/numbers_v4.json` (content unedited, font reduced). “CPU-s total” is server CPU-seconds, which include idle-spinning worker threads (D14, D21). “acc.” and “pass rate” are the `PASS_HIDDEN` rate, not the registered gate `VERIFIED`. CWR counts every `UNDECIDED` answer as wrong, so it is an upper bound. Since v3.3.1, math is scored boxed-only by default (D28), so the two math rows and the two pooled rows are equal.

Table 19: Cached generations on the E-night set (greedy, one seed). Tokens are mean completion tokens.

model	domain	n	mean tok.	CPU-s total	finish reasons
qwen3.5-2b-q4_K_M	math	26	371.0	9911	length:4, stop:22
qwen3.5-2b-q4_K_M	python	28	118.8	3625	stop:27, length:1
qwen3.5-2b-q4_K_M	rust	26	98.5	2963	stop:26

Table 20: Single unregistered arm **base**, per domain (descriptive, not a test). Pass = passed the hidden checks (PASS_HIDDEN), not the registered gate VERIFIED. Confidence = raw exp(mean log-prob). 95% cluster bootstrap CI. Cost unit: cpu_seconds. TBD = undefined (e.g. no passing item).

domain		status	n	pass rate	mean conf.	ECE	AUROC	cost / pass
math		INVALID prompt path (D24/D28); boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
python		headline	28	0.393 [0.214, 0.571]	0.896 [0.875, 0.917]	0.519 [0.392, 0.689]	0.717 [0.483, 0.917]	330 [174, 726]
rust		validated harness (D27)	26	0.231 [0.077, 0.385]	0.894 [0.882, 0.906]	0.675 [0.549, 0.813]	0.875 [0.709, 0.990]	494 [261, 1552]
math (boxed-only)		INVALID prompt path; registered boxed-only	26	0.000 [0.000, 0.000]	0.832 [0.807, 0.857]	0.832 [0.807, 0.857]	TBD [TBD, TBD]	TBD [TBD, TBD]
pooled (boxed-only math)		sensitivity	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]
pooled		sensitivity (mixed)	80	0.212 [0.125, 0.300]	0.875 [0.863, 0.886]	0.662 [0.585, 0.741]	0.806 [0.675, 0.917]	970 [629, 1648]

Table 21: Arms on the paired E-night items (percent; 95% cluster bootstrap CI). Cost unit: cpu_seconds. TBD = not run or not available.

arm	acc.	coverage	CWR	ans. acc.	cost / correct
base	21.2 [12.5, 30.0]	100.0 [100.0, 100.0]	78.8 [70.0, 87.5]	21.2 [12.5, 30.0]	970.5 [629.2, 1647.9]

Table 22: Hypothesis status for the night run. Verdicts follow the pre-registered rules; primary Holm with $m = 2$.

H	role	result	detail
H1	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H2	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H3	primary	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H4	secondary	TBD	no cross-fitted scores file (no scores file)
H5	secondary	TBD	no cross-fitted scores file (no scores file)
H6	secondary	TBD	no cross-fitted scores file (no scores file)
H7	exploratory	TBD	scored rows exist but only for unregistered single arm(s) ['base']: H1/H2/H3/H7 need paired registered arms (GL, B1, B2, B3, B5)
H8	exploratory_only_if_G1	TBD	needs G1 (no GPU slot)
H9	deferred	deferred	deferred (D8)
H10	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H11	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H12	dropped	dropped	dropped for this run (docs/DEVIATIONS.md D8)
H13	run	not refuted	
H14	only_if_E1	TBD	needs E1 (D12)
H15	offline_test	TBD	offline kill/resume test, see tests/test_run_study.py